

Sisteme de Operare - Întrebări de Examen

283 întrebări extrase din deck-ul Anki

Întrebarea 1

Q:

EXAM 16 IUNIE 2026 – Întrebarea ew01

Dați o expresie regulată care acceptă stringuri formate exclusiv din secvența "abc", repetată de cel puțin 3 ori, indiferent dacă apare cu litere mari sau mici.

A:

grep -Ei '^(abc){3,}\$' fisier

Sau, fără -i (case sensitive explicit):

grep -E '^(([Aa][Bb][Cc]){3,})\$' fisier

Expresia regulată propriu-zisă: `^(abc){3,}$`

Explicație:

- `^` și `$` = ancore (linia ÎNTREAGĂ trebuie să se potrivească, nu doar o porțiune)
- `(abc)` = grupul care conține secvența "abc"
- `{3,}` = de cel puțin 3 ori (3 sau mai multe)
- Flag-ul `-i` face matching-ul case-insensitive (acceptă "abc", "ABC", "AbC", "abcABcabc" etc.)
- Flag-ul `-E` activează expresiile regulate extinse (ERE) – necesare pentru `()` și `{n,}`

Exemple acceptate: `abcbcabcb`, `ABCABCABC`, `abcABCabc`, `abcABCabcABC`

Exemple respinse: `abcbabc` (doar 2 repetări), `abcbcabcd` (are caracter în plus), `xyzabc`

Întrebarea 2

Q:

EXAM 16 IUNIE 2026 – Întrebarea ew02

Scrieți o comandă GREP care afișează liniile de lungime impară care încep și se termină cu o vocală.

A:

grep -Ei '^[aeiou].(.)*[aeiou]\$' fisier

Sau, dacă vrem să acceptăm și liniile de lungime 1 (doar o vocală):

grep -Ei '^[aeiou]\$|^[aeiou](.)*[aeiou]\$\$' fisier

Explicație detaliată:

- ^ și \$ = linia întreagă
- [aeiou] = o vocală la început
- . = un caracter (orice)
- (..)* = perechi de caractere, repetate 0 sau mai multe ori
- [aeiou] = o vocală la final

Calculul lungimii:

- 1 (prima vocală) + 1 (caracterul oarecare) + 2k (k perechi) + 1 (ultima vocală)
- Total: 2k + 3 caractere = mereu IMPAR (3, 5, 7, 9, ...)

Flag-ul -i face matching-ul case-insensitive (acceptă și A, E, I, O, U).

Flag-ul -E activează ERE pentru a folosi () și + fără escape.

Exemple acceptate (lungime 3, 5, 7...): "aXa", "eXXXe", "iXXXXXi"

Exemple respinse: "ab" (nu se termină cu vocală), "abba" (lungime pară), "xyz" (nu începe cu vocală)

Întrebarea 3

Q:

EXAM 16 IUNIE 2026 – Întrebarea ew03

Se consideră un fișier text care conține pe fiecare linie cel puțin 4 cuvinte (secvențe nevide de litere mari sau mici) separate prin SPAȚII. Scrieți două comenzi SED care interschimbă al doilea cuvânt cu penultimul cuvânt pe fiecare linie a acestui fișier.

A:

Varianta 1 (cu -E pentru ERE):

```
sed -E 's/^(\\S+) (\\S+) (.*) (\\S+) (\\S+)$/\1 \4 \3 \2 \5/' fisier
```

Varianta 2 (echivalentă, cu [^] în loc de \\S):

```
sed -E 's/^( [^ ]+ ) ( [^ ]+ ) (.*) ( [^ ]+ ) ( [^ ]+ )$/\1 \4 \3 \2 \5/' fisier
```

Explicație:

- ^ și \$ = linia ÎNTREAGĂ
- Capturez 5 grupuri folosind paranteze ():
 - \1 = primul cuvânt (\\S+ = cel puțin un caracter ne-spațiu)
 - \2 = al doilea cuvânt (cel pe care vrem să-l mutăm)
 - \3 = totul între al doilea cuvânt și penultim (poate fi gol dacă sunt exact 4 cuvinte; .* e greedy dar regex-ul are 5 capturi obligatorii)

- \4 = penultimul cuvânt (cel pe care vrem să-l mutăm)
- \5 = ultimul cuvânt
- În înlocuire: \1 \4 \3 \2 \5 – am schimbat poziția lui \2 cu \4

Flag-ul -E (sau -r) activează expresiile regulate EXTINSE, ceea ce permite () și + fără backslash.

Exemplu: "alfa beta gamma delta epsilon" → "alfa delta gamma beta epsilon" (beta era pe poziția 2, delta era penultima – s-au interschimbat)

Întrebarea 4

Q:

EXAM 16 IUNIE 2026 – Întrebarea ew04

Scrieți o comandă AWK care afișează produsul celui de-al doilea și penultimului câmp de pe fiecare linie dacă numărul de câmpuri ale liniei e cel puțin 3.

A:

```
awk 'NF >= 3 {print $2 * $(NF-1)}' fisier
```

Explicație:

- NF = Number of Fields (numărul de câmpuri pe linia curentă, calculat automat de AWK)
- NF >= 3 = condiția (pattern-ul) care filtrează doar liniile cu cel puțin 3 câmpuri
- \$2 = al doilea câmp
- \$(NF-1) = penultimul câmp (câmpul de pe poziția NF-1)
- {print ...} = acțiunea aplicată pe liniile care îndeplinesc condiția

AWK împarte automat fiecare linie în câmpuri folosind spațiul ca delimitator implicit (FS = field separator).

Exemple:

- Linie "10 20 30 40 50" (NF=5): \$2=20, \$(NF-1)=40 → produs = 800
- Linie "5 7" (NF=2): NF < 3, nu se afișează nimic pentru această linie
- Linie "2 3 4" (NF=3): \$2=3, \$(NF-1)=3 → produs = 9 (al doilea și penultimul sunt același câmp aici, \$2)

Întrebarea 5

Q:

EXAM 16 IUNIE 2026 – Întrebarea ew05

Scrieți o linie de comandă în care programul B să se execute doar dacă programul A s-a executat cu succes.

A:

```
./a && ./b
```

Sau, dacă programele sunt în PATH:

```
A && B
```

Explicație:

- && este operatorul de logică ȘI scurt-circuit (short-circuit AND) din shell
- A && B se execută astfel:
 1. Rulează A
 2. Dacă A se termină cu exit code 0 (SUCCES), rulează și B
 3. Dacă A se termină cu exit code $\neq$ 0 (EȘEC), NU rulează B

În UNIX, convenția e:

- Exit code 0 = SUCCES
- Exit code $\neq$ 0 = EROARE (codul exact indică natura erorii)

Alte variante:

- A; B = rulează B mereu (indiferent dacă A a reușit)
- A || B = rulează B doar dacă A a EȘUAT (opusul lui &&)
- A & B = rulează A în background și pornește B imediat (nu așteaptă)

Întrebarea 6

Q:

EXAM 16 IUNIE 2026 – Întrebarea ew06

Scrieți un script Shell care citește dintr-un fișier a.txt perechi de nume de fișiere și îl șterge pe al doilea dacă primul există deja.

A:

```
#!/bin/bash
```

```
while read f1 f2; do
    if [ -e "$f1" ]; then
        rm -f "$f2"
    fi
done < a.txt
```

Explicație:

- while read f1 f2 = citește linie cu linie din stdin, fiecare linie e împărțită în două variabile (separator = spațiu/tab implicit)
- done < a.txt = redirecționează a.txt la stdin-ul lui while
- [-e "\$f1"] = testează dacă fișierul f1 EXISTĂ (oricare ar fi tipul - fișier, director, link, etc.)
- rm -f "\$f2" = șterge f2; flag-ul -f (force) evită mesaje de eroare dacă f2 nu există
- Ghilimelele duble "\$f1" și "\$f2" sunt importante pentru a gestiona nume cu spații

Exemplu de a.txt:

```
/etc/passwd /tmp/test1
fișier_inexistent.txt /tmp/test2
```

```
/var/log/syslog /tmp/test3
```

Scriptul va șterge /tmp/test1 (pentru că /etc/passwd există) și /tmp/test3, dar NU /tmp/test2 (pentru că fisier_inexistent.txt nu există).

Variantă alternativă cu test în loc de [:

```
while read f1 f2; do
    if test -e "$f1"; then
        rm -f "$f2"
    fi
done < a.txt
```

Întrebarea 7

Q:

EXAM 16 IUNIE 2026 – Întrebarea ew07

Considerând că apelul fork se execută cu succes, câte procese fiu va crea programul de mai jos? Justificați răspunsul.

```
int main() {
    while(wait(NULL) > 0)
        if(fork() == 0) {
            exit(0);
        }
    return 0;
}
```

A:

Răspuns: 0 procese fiu (zero copii).

E o capcană CLASICĂ care testează înțelegerea ordinii de evaluare a condițiilor.

Analiza pas cu pas:

1. La pornirea programului, procesul curent NU are niciun copil.
2. Se evaluează condiția while: `wait(NULL) > 0`.
3. `wait(NULL)` verifică dacă există copii de așteptat. Dacă NU există, returnează -1 cu `errno = ECHILD`.
4. `-1 > 0` evaluează la FALS.
5. Bucla while se oprește IMEDIAT, fără să intre nici măcar o dată.
6. Codul cu `fork()` din interiorul while-ului NU se execută niciodată.
7. Programul ajunge la `return 0` și se termină.

De aceea NU se creează niciun proces fiu.

Greșeala comună: să presupui că `wait("așteaptă")` implică să existe copii. În realitate, `wait()` raportează imediat eroare dacă nu sunt copii.

Dacă vroia 5 copii, codul ar fi trebuit să fie:

```
for (int i = 0; i < 5; i++) {
    if (fork() == 0) exit(0);
}
```

```
while (wait(NULL) > 0); // așteaptă toți copiii
```

Întrebarea 8

Q:

EXAM 16 IUNIE 2026 – Întrebarea ew08

Scrieți un apel exec care execută programul și argumentele date ca argumente programului ./a de mai jos (ex: ./a ls -l /etc/passwd):

```
int main(int argc, char** argv) {  
    ...  
}
```

A:

```
int main(int argc, char** argv) {  
    if (argc < 2) {  
        fprintf(stderr, "Utilizare: %s program [argumente...]\n", argv[0]);  
        return 1;  
    }  
    execvp(argv[1], &argv[1]);  
    perror("execvp"); // se executa doar daca execvp esueaza  
    return 1;  
}
```

Explicație:

Când rulezi `./a ls -l /etc/passwd`:

- argc = 4
- argv[0] = "./a" (numele programului)
- argv[1] = "ls" (programul de executat)
- argv[2] = "-l" (primul argument pentru ls)
- argv[3] = "/etc/passwd" (al doilea argument pentru ls)
- argv[4] = NULL (terminator, garantat de sistem)

De ce execvp?

- execv = ia un array de argumente (variantă cu vector)
- p = caută în PATH (deci pot folosi "ls" în loc de "/bin/ls")
- Funcția: execvp(path, argv)
 - path = argv[1] = "ls"
 - argv = &argv[1] = adresa lui argv[1], care arată spre array-ul {"ls", "-l", "/etc/passwd", NULL}

De ce &argv[1]?

În C, &argv[1] este un pointer la elementul argv[1], iar de acolo restul array-ului continuă normal (cu NULL la final). E mai compact decât să construiesc un nou array.

Variantă alternativă cu array construit explicit:

```
char **args = &argv[1];  
execvp(args[0], args);
```

Întrebarea 9

Q:

EXAM 16 IUNIE 2026 – Întrebarea ew09

Trimiteti şirul "Hello\nWorld\n" către comanda "grep -E o" folosind popen în modul scriere.

A:

```
#include <stdio.h>

int main() {
    FILE *fp = popen("grep -E o", "w");
    if (fp == NULL) {
        perror("popen");
        return 1;
    }
    fprintf(fp, "Hello\nWorld\n");
    pclose(fp);
    return 0;
}
```

Explicație:

- popen("comanda", "w") deschide un pipe spre comanda în mod SCRIERE: ce scriem prin fp ajunge ca stdin la comanda externă
- popen("comanda", "r") ar fi în mod CITIRE: am citi output-ul comenzii

Fluxul de date:

printf în fp → stdin-ul lui grep → grep filtrează → output-ul lui grep apare pe stdout-ul programului nostru

Ce va afișa pe ecran:

```
Hello
World
```

(ambele linii conțin litera 'o', deci grep le păstrează pe amândouă)

De ce pclose și nu fclose?

- pclose închide stream-ul ȘI face wait() pentru procesul copil creat de popen (evită zombi)
- fclose ar închide doar stream-ul, lăsând procesul copil ca zombie

Alternativă cu fwrite (fără formatare):

```
const char *msg = "Hello\nWorld\n";
fwrite(msg, 1, strlen(msg), fp);
```

Întrebarea 10

Q:

EXAM 16 IUNIE 2026 – Întrebarea ew10

Explicați ce se întâmplă dacă un proces încearcă să scrie într-un pipe plin al cărui capăt de scriere a fost închis de toate celelalte procese.

A:

Procesul CURENT încă are capătul de scriere DESCHIS (altfel n-ar putea scrie). Întrebarea spune că "toate celelalte procese" au închis capătul de scriere — deci doar procesul curent îl mai are deschis.

Caz 1: Pipe-ul este PLIN și capătul de citire are CEL PUȚIN UN proces care citește

- write() se BLOCHEAZĂ până când există loc în pipe (un cititor consumă date).
- Acesta este comportamentul DEFAULT (blocking mode).
- Dacă pipe-ul are mod non-blocking (setat cu O_NONBLOCK), write() returnează -1 imediat cu errno = EAGAIN sau EWOULDBLOCK.

Caz 2: Pipe-ul este PLIN și capătul de citire a fost ÎNCHIS de toate procesele

- Sistemul trimite semnalul SIGPIPE procesului care încearcă să scrie.
- Comportament default: procesul este TERMINAT (omorât) de SIGPIPE.
- Dacă SIGPIPE este IGNORAT (sau handler instalat), write() returnează -1 cu errno = EPIPE.

De ce nu interpretăm întrebarea ca "capătul de SCRIERE închis"?

Dacă TOATE capetele de scriere ar fi închise (inclusiv al procesului curent), procesul curent nu ar mai avea descriptor valid pentru pipe → write() ar eșua cu errno = EBADF.

Formularea cea mai probabilă a întrebării:

"Capătul de CITIRE a fost închis de toate procesele" → atunci scrierea declanșează SIGPIPE / EPIPE (cazul 2).

Notă: comportamentul cu SIGPIPE este o sursă frecventă de "crash-uri misterioase" în programe care folosesc pipe-uri sau sockets.

Întrebarea 11

Q:

EXAM 16 IUNIE 2026 – Întrebarea ew11

Discutați durata de execuție a programului de mai jos în funcție de tipul și permisiunile fișierului abc.

```
int main() {
    int f;
    char c;
    f = open("abc", O_RDONLY);
    if(f >= 0) {
        read(f, &c, 1);
    }
}
```

```

        close(f);
    }
    return 0;
}

```

A:

Durata depinde semnificativ de TIPUL fişierului abc:

1. Fişier NORMAL (regular file) cu permisiune de citire
 - DURATĂ FOARTE SCURTĂ (microsecunde)
 - open, read 1 octet, close. Totul aproape instantaneu.
2. Fişier NORMAL fără permisiune de citire pentru utilizator
 - DURATĂ FOARTE SCURTĂ
 - open returnează -1 (EACCES), sare peste if, exit.
3. Fişier care NU EXISTĂ
 - DURATĂ FOARTE SCURTĂ
 - open returnează -1 (ENOENT), sare peste if, exit.
4. FIFO (named pipe) fără scriitor activ
 - DURATĂ INFINITĂ (programul se blochează la open!)
 - open(O_RDONLY) pe un FIFO BLOCHEAZĂ până când un alt proces deschide FIFO-ul pentru SCRIERE (O_WRONLY).
 - Dacă niciun scriitor nu apare, programul rămâne blocat la nesfârşit.
5. FIFO cu scriitor activ dar fără date trimise
 - DURATĂ LUNGĂ (blocaţi la read)
 - open trece (scriitorul există), dar read se blochează până vin date.
6. FIFO cu scriitor activ şi date disponibile
 - DURATĂ SCURTĂ — citeşte 1 octet şi termină.
7. DIRECTOR
 - Pe Linux modern, open pe director cu O_RDONLY reuşeşte, dar read returnează -1 (EISDIR).
 - Durată scurtă.
8. DISPOZITIV CHARACTER/BLOC (ex: /dev/random, /dev/sda)
 - Depinde de dispozitiv. /dev/zero răspunde instant, /dev/random poate aştepta entropie.

9. SOCKET sau alt tip special

→ De obicei open eșuează sau read are comportament specific.

Punctul cel mai important: FIFO-urile pot bloca la open() — o capcană frecventă în programe care nu știu tipul fișierului.

Întrebarea 12

Q:

EXAM 16 IUNIE 2026 – Întrebarea ew12

Protejați secțiunea critică din programul de mai jos.

```
int n = 0;

void* f(void* v) {
    if(n < 500) {
        n++;
    }
    return NULL;
}

int main() {
    pthread_t t;
    pthread_create(&t, NULL, f, NULL);
    pthread_join(t, NULL);
    return 0;
}
```

A:

```
#include <pthread.h>

int n = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

void* f(void* v) {
    pthread_mutex_lock(&m);
    if (n < 500) {
        n++;
    }
    pthread_mutex_unlock(&m);
    return NULL;
}

int main() {
    pthread_t t;
    pthread_create(&t, NULL, f, NULL);
    pthread_join(t, NULL);
    pthread_mutex_destroy(&m);
    return 0;
}
```

Explicație: secțiunea critică este blocul `if (n < 500) { n++; }`

De ce este zonă critică?

1. Operația `n++` NU este atomică – în spate are 3 pași:

- citește `n` din memorie în registru
- adună 1 în registru
- scrie registrul înapoi în memorie

Două thread-uri pot citi aceeași valoare, o incrementează și o scriu – o incrementare se pierde (LOST UPDATE).

2. Combinația `if (n < 500)` urmată de `n++` are RACE CONDITION:

- Thread A citește `n=499`, evaluează condiția (`true`)
- Thread B citește `n=499`, evaluează condiția (`true`)
- Ambele fac `n++` → `n` ajunge la 501, depășind limita

Mutex-ul rezolvă ambele probleme: doar UN thread la un moment dat poate intra între `lock` și `unlock`.

Observație: în codul dat, e doar UN thread (`pthread_create` creat o singură dată). În acel caz mutex-ul nu e strict necesar – dar întrebarea cere PROTECȚIA secțiunii critice, deci o adăugăm pentru cazul general (când există mai multe thread-uri).

Variantă cu `PTHREAD_MUTEX_INITIALIZER` (inițializare statică, fără `pthread_mutex_init`). Variantă alternativă cu inițializare dinamică:

```
pthread_mutex_t m;  
pthread_mutex_init(&m, NULL);  
// ... folosire ...  
pthread_mutex_destroy(&m);
```

Întrebarea 13

Q:

EXAM 16 IUNIE 2026 – Întrebarea ew15

Cum ar fi afectată funcționarea unui program dacă toate mutex-urile s-ar înlocui cu `rwlock`-uri și toate apelurile `mutex_lock` s-ar înlocui cu `rwlock_wrlock`? Justificați răspunsul.

A:

Programul ar funcționa PRACTIC IDENTIC – `pthread_rwlock_wrlock` se comportă la fel ca `pthread_mutex_lock`: ambele oferă acces EXCLUSIV (un singur thread la un moment dat).

Funcționalitatea `rwlock`-urilor:

- `rwlock_rdlock` = read lock = MAI MULȚI cititori pot avea concurent (lock partajat)
- `rwlock_wrlock` = write lock = UN SINGUR thread, exclusiv (la fel ca un mutex)

Deoarece toate apelurile au fost înlocuite cu `wrlock` (nu `rdlock`), se păstrează semantica de excludere mutuală.

DIFERENȚE MICI care pot afecta programul:

1. Performanță ușor mai slabă:

rwlock-urile au structuri interne mai complexe (țin evidența cititorilor și scriitorilor), deci sunt cu puțin mai lente decât mutex-urile.

2. Posibile probleme de starvation:

Implementările rwlock pot prioritiza diferit. În unele implementări, un wrlock care așteaptă poate fi blocat la nesfârșit dacă cititorii continuă să vină. Aici nu mai are sens (toate sunt wrlock), dar dacă se amestecă cu rdlock în viitor, pot apărea probleme.

3. Memorie:

rwlock-urile ocupă mai mult spațiu decât mutex-urile.

4. Variabile condiționale:

pthread_cond_wait funcționează DOAR cu pthread_mutex_t, NU cu rwlock. Dacă programul folosește variabile condiționale, înlocuirea va FACE PROGRAMUL SĂ NU MAI COMPILAZE (sau crashes la runtime).

ATENȚIE – CONFUZIE FRECVENTĂ:

Dacă întrebarea ar fi cerut înlocuirea cu rwlock_RDLOCK (read lock), atunci AR FI APĂRUT race conditions, pentru că rdlock permite mai mulți cititori simultan, și aceștia pot modifica fără protecție resursa partajată.

Concluzie: cu wrlock, programul funcționează corect (eventual ușor mai lent), DAR e o alegere proastă din punct de vedere al designului – rwlock-urile au sens DOAR când distingi între citiri (multe) și scrieri (puține).

Întrebarea 14

Q:

EXAM 16 IUNIE 2026 – Întrebarea ew16

Dați valori distincte pentru T, N1, N2, N3 (toate strict pozitive) pentru care programul de mai jos se termină corect.

```
pthread_barrier_t b1, b2, b3;
```

```
void* f1(void* a) { pthread_barrier_wait(&b1); return NULL; }
void* f2(void* a) { pthread_barrier_wait(&b2); return NULL; }
void* f3(void* a) { pthread_barrier_wait(&b3); return NULL; }
```

```
int main() {
    int T, N1, N2, N3;
    pthread_t t[T][3];
    pthread_barrier_init(&b1, NULL, N1);
    pthread_barrier_init(&b2, NULL, N2);
    pthread_barrier_init(&b3, NULL, N3);

    for(int i=0; i<T; i++) {
        pthread_create(&t[i][0], NULL, f1, NULL);
```

```

        pthread_create(&t[i][1], NULL, f2, NULL);
        pthread_create(&t[i][2], NULL, f3, NULL);
    }
    // join-uri + destroy-uri
    return 0;
}

```

A:

Valori distincte care funcționează: $T = 12$, $N1 = 2$, $N2 = 3$, $N3 = 4$

Explicație:

Codul creează:

- T thread-uri care apelează $f1$ (deci T thread-uri ajung la bariera $b1$)
- T thread-uri care apelează $f2$ (T ajung la $b2$)
- T thread-uri care apelează $f3$ (T ajung la $b3$)

Bariera b_i se eliberează DOAR când exact N_i thread-uri ajung la ea. Apoi se RESETEAZĂ și așteaptă încă N_i thread-uri etc.

Deci, pentru ca TOATE thread-urile să poată trece bariera și programul să se termine:

- T trebuie să fie MULTIPLU al lui $N1$ (altfel un grup de $< N1$ thread-uri va rămâne blocat)
- T trebuie să fie multiplu al lui $N2$
- T trebuie să fie multiplu al lui $N3$

Condiția: $T = k1 \cdot N1 = k2 \cdot N2 = k3 \cdot N3$ pentru anumite $k1, k2, k3$ întregi pozitivi.

Alegerea $T = 12$, $N1 = 2$, $N2 = 3$, $N3 = 4$:

- $12 / 2 = 6 \rightarrow b1$ se eliberează de 6 ori, $6 \cdot 2 = 12$ thread-uri trec ✓
- $12 / 3 = 4 \rightarrow b2$ se eliberează de 4 ori, $4 \cdot 3 = 12$ thread-uri trec ✓
- $12 / 4 = 3 \rightarrow b3$ se eliberează de 3 ori, $3 \cdot 4 = 12$ thread-uri trec ✓
- Toate valorile sunt distincte ($2 \neq 3 \neq 4 \neq 12$) ✓

Alte combinații care funcționează:

- $T = 24$, $N1 = 2$, $N2 = 3$, $N3 = 4$ (multiplii pari)
- $T = 6$, $N1 = 2$, $N2 = 3$, $N3 = 6$ (atenție, $N3 = T$)
- $T = 30$, $N1 = 2$, $N2 = 3$, $N3 = 5$

Dacă alegem rău (de ex. $T = 5$, $N1 = 2$):

- $5 / 2 = 2$ rest 1 $\rightarrow$ primele 2 grupe de 2 thread-uri trec, dar al 5-lea thread rămâne BLOCAT la bariera $b1$ la nesfârșit $\rightarrow$ DEADLOCK.

Întrebarea 15

Q:

EXAM 16 IUNIE 2026 – Întrebarea ew17

Ce puteți face ca programator pentru a preveni deadlock-urile cauzate de blocarea unor mutex-uri? Justificați răspunsul.

A:

Există mai multe strategii. Cea mai puternică și folosită:

1. ORDINE FIXĂ DE BLOCARE (lock ordering)

Stabilim o ordine globală a mutex-urilor și mereu le blocăm ÎN ACEEAȘI ORDINE în toate thread-urile.

Exemplu:

- Mutex A, B, C – alegem ordinea $A < B < C$
- Orice thread care vrea să blocheze două mutex-uri trebuie să le blocheze în această ordine: A apoi B (nu B apoi A), B apoi C (nu C apoi B), etc.

De ce funcționează?

Deadlock-urile apar când se formează un CICLU de așteptare:

- Thread 1 are A, vrea B
- Thread 2 are B, vrea A
- Ciclul $A \rightarrow B \rightarrow A$ blochează totul.

Cu ordine fixă, ciclul nu se poate forma: dacă toți blochează în ordinea $A \rightarrow B$, atunci cel care are A va elibera A înainte ca cineva să-l ceară.

2. TRY-LOCK CU RETRY (pthread_mutex_trylock)

Encearcăm să blocăm cu `pthread_mutex_trylock` (nu blochează – returnează imediat dacă mutex-ul e ocupat). Dacă eșuăm, eliberăm tot ce am deja și încercăm de la zero.

```
if (pthread_mutex_lock(&m1) == 0) {  
    if (pthread_mutex_trylock(&m2) == 0) {  
        // am ambele, fac treaba  
        pthread_mutex_unlock(&m2);  
    } else {  
        pthread_mutex_unlock(&m1);  
        // încerc din nou mai târziu  
    }  
}
```

3. TIMEOUT PE BLOCARE (pthread_mutex_timedlock)

În loc de blocaj permanent, încercăm pe o durată limitată. După timeout, eliberăm și reluăm.

4. EVITĂ BLOCAJE IMBRICATE

Dacă e posibil, nu țină un mutex în timp ce încerci să obții altul. Eliberează A înainte să ceri B.

5. ALGORITM BANKER (Dijkstra)

La nivel de sistem, înainte să acordăm o resursă, verificăm dacă acordarea ar putea duce la stare nesigură. Folosit mai mult la nivel de SO, nu de programator obișnuit.

6. DETECTARE ȘI RECUPERARE

Un watchdog detectează ciclurile de așteptare și forțează un proces/thread să elibereze resursele. Costisitor – folosit doar în sisteme critice.

În practică, cea mai folosită soluție e ORDINEA FIXĂ: simplă, eficientă, fără overhead.

Întrebarea 16

Q:

EXAM 16 Iunie 2026 – Întrebarea ew18

Care este diferența dintre politicile de planificare SJF și STCF?

A:

Ambele aleg jobul cu cea mai mică durată, dar diferă prin PREEMPTIE și prin ce durată consideră:

SJF (Shortest Job First):

- NON-PREEMPTIV
- Decide o singură dată, când un job se termină sau pleacă
- Alege jobul cu DURATA TOTALĂ cea mai scurtă din coada de Ready
- Odată ce un job pornește, rulează NEÎNTRERUPT până se termină
- Dacă apare un job nou mai scurt în timpul execuției, e ignorat până când jobul curent termină

STCF (Shortest Time to Completion First):

- PREEMPTIV (versiunea preemptivă a lui SJF)
- Re-evaluează la FIECARE eveniment (job nou sosit, job terminat)
- Alege jobul cu TIMPUL RĂMAS cel mai scurt
- Poate ÎNTRERUPE un job în execuție dacă apare unul cu timp rămas mai mic

EXEMPLU CONCRET:

La $t=0$ vine A cu durată 10. Pornim A.

La $t=2$ vine B cu durată 3.

Cu SJF:

- A continuă să ruleze (e deja pornit, SJF nu preempt).
- A se termină la $t=10$.

- B rulează de la $t=10$ la $t=13$.
- Timp mediu de așteptare: $A=0$, $B=8 \rightarrow$ media $(0+8)/2 = 4$
- Turnaround: $A=10$, $B=11 \rightarrow$ media 10.5

Cu STCF:

- La $t=2$ verificăm: A are 8 rămase, B are 3.
- $3 < 8 \rightarrow$ STCF întrerupe A, pornește B.
- B se termină la $t=5$.
- A reia de la $t=5$, se termină la $t=13$.
- Timp mediu de așteptare: $A=3$ (între $t=2$ și $t=5$), $B=0 \rightarrow$ media 1.5
- Turnaround: $A=13$, $B=3 \rightarrow$ media 8

STCF dă rezultate mai bune, dar costă mai mult (overhead de schimbare contexte) și poate cauza STARVATION pentru job-uri lungi care sunt mereu preemptiv.

ALTĂ PERSPECTIVĂ:

SJF = SRJ (Shortest Remaining Job) non-preemptiv

STCF = SRT (Shortest Remaining Time) preemptiv

Întrebarea 17

Q:

EXAM 16 IUNIE 2026 – Întrebarea ew19

Un proces are următoarea listă de zone de memorie nealocată: 32, 44, 98. Cum va arăta lista după două apeluri malloc(33) știind că se folosește algoritmul Worst Fit?

A:

Răspuns: [32, 44, 32] (sau, ordonat crescător: [32, 32, 44])

Algoritmul Worst Fit:

La fiecare cerere de alocare, alege CEA MAI MARE zonă liberă suficient de mare. După alocare, zona rămasă (diferența) rămâne disponibilă.

Logica: lăsând zone mari, șansa ca alocările viitoare să încapă într-un loc fără să fragmenteze e mai mare (în teorie). În practică, Worst Fit performă slab pentru că epuizează rapid zonele mari.

Pas cu pas:

Stare inițială: [32, 44, 98]

Apel 1: malloc(33)

- Zonele disponibile suficient de mari: 44 (≥ 33) și 98 (≥ 33). 32 e prea mică.
- Worst Fit alege CEA MAI MARE → 98
- $98 - 33 = 65$ rămas în acea zonă
- Stare: [32, 44, 65]

Apel 2: malloc(33)

- Zonele suficiente: 44 (≥ 33) și 65 (≥ 33). 32 încă prea mică.
- Worst Fit alege CEA MAI MARE → 65
- $65 - 33 = 32$ rămas
- Stare: [32, 44, 32]

Observație despre Worst Fit:

- A folosit zonele 98 și 65 pentru două alocări de 33 fiecare.
- Acum cele 3 zone (32, 44, 32) sunt prea mici pentru altă alocare mare.
- Compară cu First Fit: ar fi folosit 44 prima dată (rămâne 11), apoi 98 (rămâne 65). Rezultat: [32, 11, 65]
- Compară cu Best Fit: ar fi folosit 44 ambele dați... nu, $44 - 33 = 11$ e prea mic la a doua. Deci 44 ($\Rightarrow 11$) și apoi 98 ($\Rightarrow 65$). Rezultat: [32, 11, 65]

Întrebarea 18

Q:

EXAM 16 Iunie 2026 – Întrebarea ew20

Cum diferă un link hard de unul simbolic?

A:

Hard link vs Symbolic link (soft link):

1. CE INDICĂ:

- Hard link: indică spre același I-NODE ca fișierul original. Practic, sunt nume diferite pentru același fișier fizic.
- Symbolic link: este un fișier separat care conține CALEA (un string) către fișierul țintă.

2. TRAVERSAREA SISTEMELOR DE FIȘIERE:

- Hard link: NU poate traversa partiții/sisteme de fișiere (i-node-urile sunt unice doar într-un sistem de fișiere).
- Symbolic link: POATE traversa partiții — fiindcă stochează doar o cale text.

3. ȘTERGEREA ȚINTEI:

- Hard link: fișierul EXISTĂ atât timp cât există măcar un hard link (există un contor de hard links în i-node). Ștergerea fișierului original nu afectează celelalte hard link-uri.
- Symbolic link: dacă fișierul țintă e șters, link-ul devine "dangling" (rupt) — încă există, dar accesarea lui dă eroare ENOENT.

4. SPRE DIRECTOARE:

- Hard link: De obicei INTERZIS pentru directoare (cauzează cicluri în arborele de fișiere). Excepție: . și .. sunt hard link-uri speciale.
- Symbolic link: PERMIS spre orice (fișiere, directoare, alte link-uri).

5. COMENZI DE CREARE:

- Hard link: ln target nume_link
- Symbolic link: ln -s target nume_link

6. VIZUALIZARE CU ls -l:

- Hard link: apare ca un fișier obișnuit cu același tip și permisiuni.
- Symbolic link: apare cu prefix l (link), arată 'link -> target'.

7. DIMENSIUNE:

- Hard link: nu adaugă spațiu (e o intrare în director, foarte mică).
- Symbolic link: ocupă spațiul necesar pentru a stoca calea text (de obicei foarte puțin).

8. PERMISIUNI:

- Hard link: au exact aceleași permisiuni ca fișierul țintă (sunt același i-node).
- Symbolic link: au propriile permisiuni, dar accesul real depinde de permisiunile țintei.

EXEMPLU PRACTIC:

file.txt → are i-node 12345 (conține datele "Hello")

Dacă fac: ln file.txt hard.txt

- hard.txt are ȘI EL i-node 12345 (același!)
- Acum sunt 2 hard links pentru aceleași date.
- Ștergerea file.txt → datele rămân (hard.txt încă există).

Dacă fac: ln -s file.txt soft.txt

- soft.txt e un nou fișier mic care conține textul "file.txt"
- Ștergerea file.txt → soft.txt devine "broken link", citirea lui dă eroare.

USE CASE:

- Hard link: backup-uri eficiente (nu duplică datele), versiuni multiple ale aceluiași fișier.
- Symbolic link: scurtături, gestiunea de versiuni (ex: python → python3.9), traversarea partițiilor.

Întrebarea 19

Q:

Dați trei expresii regulate care se potrivesc cu orice număr nenegativ multiplu de 5.

A:

- 1) `"\<[0-9]*[05]\>"`
- 2) `"\<[0-9]+(0|5)\>"`
- 3) `"^[0-9]*[05]$"`

Explicație:

- Un număr e multiplu de 5 dacă se termină în 0 sau 5
- `\<` și `\>` = limite de cuvânt (word boundaries) — prinde numărul ca entitate
- `[0-9]*` = zero sau mai multe cifre înainte
- `[05]` = ultima cifră trebuie să fie 0 sau 5
- `^...$` = a treia variantă cere ca ÎNTREAGA LINIE să fie un astfel de număr

Exemple care match-uiesc: 0, 5, 10, 15, 20, 25, 100, 1235

Exemple care NU match-uiesc: 1, 7, 23, 111

Întrebarea 20

Q:

Dați cinci comenzi GREP care afiseaza toate liniile dintr-un fisier care contin litera "a" mare sau mic.

A:

```
grep "[Aa]" file
grep -E "[Aa]" file
grep -i "a" file
grep -E "A|a" file
grep -iw "a" file
```

Observații:

- [Aa] = clasă de caractere care matchează A sau a (cea mai simplă)
 - i = case insensitive (ignoră majuscule/minuscule)
 - A|a cu -E (extended regex) folosește | ca alternativă
 - w cere ca match-ul să fie cuvânt întreg
-

Întrebarea 21

Q:

Scrieti doua comenzi SED care afiseaza dintr-un fisier doar liniile care nu contin cifra 7.

A:

sed -E '/7/d' file

sed -n '/7!/p' file

Explicație:

- /7/d → șterge liniile care conțin 7 (afișează restul)
 - /7!/p → cu -n (suprimă print default), tipărește doar liniile care NU conțin 7
-

Întrebarea 22

Q:

Scrieti o comanda AWK care afiseaza suma penultimului camp al tuturor liniilor.

A:

Presupunând că separatorul este spațiu:

```
awk '{ sum += $(NF-1) } END { print sum }' fisier
```

Explicație:

- NF = numărul de câmpuri pe linie
 - \$(NF-1) = penultimul câmp
 - sum acumulează pentru fiecare linie
 - END { print sum } afișează la final
-

Întrebarea 23

Q:

Cum puteti redirecta in linia de comanda iesirea de eroare prin pipe inspre un alt program?

A:

```
command 2>&1 | other_program
```

Explicație: "2>&1" redirectează stderr (descriptor 2) la stdout (descriptor 1), iar apoi pipe-ul preia stdout-ul combinat.

Sau, dacă vrem doar stderr:

```
command 2>&1 >/dev/null | other_program
```

Întrebarea 24

Q:

Scrieti un script Shell UNIX care afisează toate argumentele din 1 inia de comanda fara a folosi FOR.

A:

```
#!/bin/bash
```

```
while [ $# -gt 0 ]; do
```

```
    echo $1
```

```
    shift
done
```

Explicație:

- \$# = numărul de argumente rămase
 - \$1 = primul argument (NU \$0, care e numele scriptului!)
 - shift mută argumentele la stânga (\$2 devine \$1, etc.)
-

Întrebarea 25

Q:

Desenati ierarhia proceselor create de codul e mai jos, incluzand procesul parinte.

```
for(i=0; i<3; it+) {
fork();
execlp("ls", "Ls", "/", NULL);
```

A:

Cod original (cu OCR-uri):

```
for(i=0; i<3; i++) {
    fork();
```

```
    execlp("ls", "ls", "/", NULL);  
}
```

Dacă `execlp` REUȘEȘTE: procesul curent e înlocuit cu `ls`, NU se mai întoarce în buclă.

Ierarhia:

P face `fork()` → apare C1

Apoi P intră în `execlp` și DEVINE `ls` (nu mai continuă bucla)

C1 intră și el în `execlp` și DEVINE `ls`

Rezultat: P creează DOAR un copil C1, apoi ambele devin `ls`.

P → C1

(Dacă `execlp` ar EȘUA, atunci ar continua bucla – 7 procese în total.)

Întrebarea 26

Q:

Adăugați codul C necesar pentru ca fișierul `b.txt` să fie suprascris cu conținutul fișierului `a.txt` din instrucțiunea de mai jos.

```
execlp("cat", "cat", "a.txt", NULL);
```

A:

Folosim `dupa` pentru a redirecta `stdout` în fișierul `b.txt`:

```
int fd = open("b.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);  
dup2(fd, 1); // stdout (descriptor 1) merge acum în b.txt  
close(fd);  
execlp("cat", "cat", "a.txt", NULL);
```

Explicație flag-uri `open`:

- `O_WRONLY`: deschidem pentru scriere
 - `O_CREAT`: creează fișierul dacă nu există
 - `O_TRUNC`: golește fișierul existent (suprascriere)
 - `0644`: permisiuni (`rw-r--r--`)
-

Întrebarea 27

Q:

De ce nu e recomandat sa comunicati bidirectional printr-un singur

FIFO?

A:

Răspuns:

Nu suntem siguri de ordinea în care procesele vor scrie în FIFO, ceea ce poate cauza coruperea datelor.

Să presupunem că procesul A vrea să scrie "abc" și apoi procesul B "123".

Dacă A pierde CPU-ul în timpul scrierii, putem avea ceva de genul:

a1bc23

Întrebarea 28

Q:

Cate FIFO-uri poate deschide un process daca nu sunt și nici nu vor

fi folosite vreodata de vreun alt proces?

A:

Răspuns:

Un proces poate deschide cel mult un FIFO dacă nu există alt proces care să folosească acel FIFO, deoarece funcția open așteaptă până când un alt proces deschide FIFO-ul pentru capătul complementar.

Întrebarea 29

Q:

Când ați folosi un process în locul unui thread?

A:

Folosesc procese în loc de thread-uri când:

- Vreau izolare de memorie (un crash într-un proces nu afectează celelalte)
- Programele trebuie să fie complet independente (alte executabile via exec)
- Vreau să rulez pe mașini diferite (procesele pot fi distribuite, thread-urile nu)
- Lucrez cu cod care nu e thread-safe (biblioteci vechi)
- Securitate: procesele au granițe clare de securitate (spații de adrese diferite)
- Vreau să rulez programe externe (ls, grep, etc.) — necesită fork + exec

Întrebarea 30

Q:

Ce este o "secțiune critica"?

A:

Răspuns:

O secțiune critică este o porțiune de cod executată de mai multe thread-uri și care conține o resursă critică (mai multe thread-uri pot modifica acea resursă).

Întrebarea 31

Q:

De ce un thread trebuie să verifice condiția la ieșirea din apelul `pthread_cond_wait`?

`pthread_cond_wait` call?

A:

Răspuns:

Un thread ar trebui să verifice condiția după întoarcerea din `pthread_cond_wait` pentru că și alte thread-uri pot aștepta pe aceleași variabile condiționale și pot fi notificate înaintea thread-ului curent, ceea ce ar putea face ca resursa cerută să nu mai fie disponibilă în cantitatea necesară.

Întrebarea 32

Q:

Care va fi efectul înlocuirii apelurilor la `pthread_mutex_lock` cu apeluri la `pthread_rwlock_rdlock`?

`pthread_mutex_lock` with calls to `pthread_rwlock_rdlock`?

A:

Răspuns:

Dacă înlocuim `pthread_mutex_lock` cu `pthread_rwlock_rdlock`, nu vom mai proteja resursa critică, deoarece `rdlock` permite mai multor cititori în același timp, dar nu este sigur că acești cititori nu vor modifica resursa.

Dacă resursa nu este modificată de cititori (doar citirea este necesară de către thread-uri), atunci înlocuirea va duce la o performanță mai bună.

Întrebarea 33

Q:

Care e efectul apelului la `pthread_barrier_wait` pentru o bariera inițializată cu 1?

`pthread_barrier_wait` on a barrier initialized with 1?

A:

Răspuns:

Nu există niciun efect la așteptarea pe o barieră cu valoare inițială 1, deoarece thread-ul care apelează `wait` va face bariera să se decrementeze la 0 și va permite thread-ului să treacă, resetând bariera înapoi la 1.

Întrebarea 34

Q:

Cum puteți incrementa valoarea unui semafor?

A:

Răspuns:

Pentru a incrementa valoarea unui semafor putem folosi funcția `sem_post`.

Întrebarea 35

Q:

Ce puteți face ca programator pentru a preveni deadlock-urile?

A:

Metode pentru prevenirea deadlock-urilor:

1. Ordine consistentă de blocare: stabiliți o ordine globală a resurselor (ex: numerotare) și blocați-le mereu în aceeași ordine.
2. Folosiți timeout-uri (`pthread_mutex_timedlock`) pentru a renunța la încercări lungi.
3. Tehnica trylock: încercați `pthread_mutex_trylock` – dacă eșuează, eliberați tot și încercați din nou.
4. Detectare și recuperare: folosiți algoritmi care detectează deadlock-urile (ex: algoritmul bancherului) și recuperează procesele.
5. Evitați nested locks atunci când e posibil.

Întrebarea 36

Q:

Prin ce tranziție de stare va trece un process când citește dintr-un fisier?

A:

Răspuns:

Procesul va trece în starea de așteptare (WAIT) până când conținutul fișierului cerut este încărcat în memorie de pe disc.

Întrebarea 37

Q:

Ce conține superblocul unui disc Linux?

A:

Răspuns:

Superblocul de pe un disc Linux conține metadate despre organizarea sistemului de fișiere. Cunoaște unde începe fiecare partiție, dimensiunea discului etc.

Întrebarea 38

Q:

Se poate crea un link hard spre un fisier aflat pe o altă partiție ? Justificați răspunsul.

A:

Răspuns:

Nu putem avea un hard link către un fișier de pe altă partiție, deoarece un hard link ține evidența i-node-ului fișierului. Din moment ce i-node-urile sunt disponibile doar în interiorul aceleiași partiții, nu putem avea un hard link către un fișier de pe altă partiție pentru că nu avem acces la numărul i-node-ului.

Întrebarea 39

Q:

Dați o expresie regulată care acceptă orice secvență de lungime pară de cuvinte (formate din litere) separate prin spații, cu condiția ca pentru fiecare cuvânt să lungimea și poziția în secvență să fie ambele pare sau impare. Numărarea cuvintelor începe de la 1. Ex: al 5-lea cuvânt trebuie să aibă lungime impară, iar al 16-lea să aibă lungime pară.

A:

Regex pentru secvențe pare de cuvinte cu condiția poziție-paritate=lungime-paritate:

```
^[a-z]([a-z][a-z])* [a-z][a-z]([a-z][a-z])* )*[a-z]([a-z][a-z])* [a-z][a-z]([a-z][a-z])*$
```

Explicație:

- `[a-z]([a-z][a-z])*` = cuvânt cu lungime IMPARĂ (1, 3, 5, ...)
- `[a-z][a-z]([a-z][a-z])*` = cuvânt cu lungime PARĂ (2, 4, 6, ...)
- Alternanță (impar) (par) (impar) (par) ... — total de perechi → lungime pară

Întrebarea 40

Q:

Dați trei comenzi GREP care afișează dintr-un fișier liniile formate exclusiv dintr-o secvență nevidă de litere și cifre alternativ (ex: a0g sau 1r5m).

A:

Answer:

```
grep -E -i "^(^([0-9]?([a-z][0-9])*)$|^([a-z]?([0-9][a-z])*)$)"
```

```
grep -E "^(^([0-9]?([a-zA-Z][0-9])*)$|^([a-zA-Z]?([0-9][a-zA-Z])*)$)"
```

```
grep -E -i -v "[a-z][a-z][0-9][0-9]" a.txt
```

```
grep -E "((^\\d?([a-zA-Z]\\d)*$)|(^[a-zA-Z]?\\d[a-zA-Z]*$))" file.txt
```

```
grep -E -i "((^\\d?([a-z]\\d)*$)|(^[a-z]?\\d[a-z]*$))" file.txt
```

```
grep -E "((^([0-9]?([a-zA-Z][0-9])*)$|^([a-zA-Z]?([0-9][a-zA-Z])*)$))" file.txt
```

Întrebarea 41

Q:

Scrieți două comenzi SED care afișează liniile unui fișier ștergând prima secvență nevidă de litere mici.

A:

```
sed -E 's/([a-z]+)/'
```

```
sed -E 's/([a-z][a-z]*)/'
```

Answer:

```
sed -E 's/[a-z]+/'
```

```
sed 's/[a-z]\+/'
```

Întrebarea 42

Q:

Scrieți o comandă AWK care afișează suma tuturor numerelor dintr-un fișier text ale cărui linii sunt formate din secvențe de cifre separate prin spații.

A:

```
awk 'BEGIN {sum = 0} {for (i = 1; i <= NF; i++) sum += $i} END { print sum}'
```

Answer:

```
awk "{for(i = 1; i <= NF; i++) SUM += $i} END {print SUM}"
```

Întrebarea 43

Q:

Dați trei moduri de a afla dimensiunea unui fișier în linia de comandă Linux.

A:

Răspuns:

1) du

Exemplu: du fisier

2) ls -s

Exemplu: ls -s fisier

3) wc -c fisier

Alternative:

ls -lh fisier

du -h fisier

stat -c %d fisier

Întrebarea 44

Q:

Scrieți o condiție Shell UNIX care verifică dacă un fișier există și utilizatorul curent are vreo permisiune asupra lui.

A:

Răspuns:

```
[ -e fisier ] && [ -r fisier -o -w fisier -o -x fisier ]
```

Sau echivalent:

```
[ -e fisier ] && { [ -r fisier ] || [ -w fisier ] || [ -x fisier ]; }
```

Întrebarea 45

Q:

Desenați ierarhia proceselor create de codul de mai jos, incluzând procesul părinte.

```
for(i=0; i<3; i++) {  
    if(fork() > 0) {  
        wait(0);  
        wait(0);  
        exit(0);  
    }  
}exam11july2023.txt
```

A:

Ierarhia este un LANȚ: $P \rightarrow C1 \rightarrow C2 \rightarrow C3$

Explicație pas cu pas:

- i=0: P face fork → C1.

P (fork > 0) intră în if și face wait(0), wait(0), exit. P se termină.

C1 (fork == 0) NU intră în if, continuă bucla.

- i=1, în C1: fork → C2.
C1 (fork > 0) face wait, wait, exit.
C2 (fork == 0) continuă bucla.
- i=2, în C2: fork → C3.
C2 face wait, wait, exit.
C3 continuă bucla, iese (i devine 3) și se termină.

Observație: codul are un bug — wait(0) e apelat de două ori, dar fiecare proces are doar UN copil, nu doi. Al doilea wait va returna -1 (errno = ECHILD).

Întrebarea 46

Q:

De ce descriptorul de fișier returnat de popen trebuie închis cu pclose și nu cu fclose?

A:

Răspuns:

La implementarea ei, popen creează un proces nou, după care trebuie să așteptăm. Dacă am folosi fclose, nu am aștepta procesul, deci trebuie să folosim pclose, deoarece aceasta așteaptă terminarea procesului și returnează statusul de ieșire al comenzii.

Deoarece popen deschide un nou proces care trebuie închis cu pclose. Pclose închide pointer-ul de fișier, așteaptă terminarea procesului asociat și returnează statusul terminării, în timp ce fclose ar închide doar stream-ul de fișier, fără a îndeplini celelalte sarcini.

Întrebarea 47

Q:

Când ați folosi execv în locul de execl?

```
execv instead of execl?  
exam11july2023
```

A:

Răspuns:

Aș folosi execv în loc de execl atunci când vreau să am argumentele comenzii de executat stocate într-o variabilă. De exemplu, ar fi util când vrem să folosim argumentele din linia de comandă care sunt stocate în char **argv.

Aș folosi execv când trebuie să transmit un array de argumente programului nou, util când numărul de argumente este stocat, permițând transmiterea argumentelor procesului nou.

Întrebarea 48

Q:

Dați trei apeluri de funcții care asigură excludere mutuală.

A:

Răspuns:

- 1) `pthread_mutex_lock(&mtx)`
- 2) Folosind semafoare binare: `sem_wait(&sem)`
- 3) `pthread_rwlock_wrlock(&rwlock)`

Întrebarea 49

Q:

Care va fi efectul înlocuirii apelurilor la `pthread_mutex_lock` cu apeluri la `sem_post`?

`pthread_mutex_lock` with calls to `sem_post`?

A:

Răspuns:

Haos. `sem_post` incrementează valoarea semaforului, deci nu va bloca niciun thread de la a intra într-o secțiune critică doar prin el însuși. Nu vom avea niciun mecanism de prevenire pe secțiunea critică, așa că putem accesa concurent locații de memorie, rezultând în răspunsuri nedorite.

Înlocuirea `pthread_mutex_lock` cu `sem_post` ar duce la haos, deoarece `sem_post` incrementează semaforul, permițând oricărui thread să intre în secțiunea critică, eșuând astfel în prevenirea accesului concurent la resursele partajate. Acest lucru ar putea duce la race conditions și corupere de date. Mai mult, dacă înlocuim doar `pthread_mutex_lock` dar păstrăm `pthread_mutex_unlock`, va rezulta într-un comportament nedefinit, deoarece deblocăm un mutex care nu a fost niciodată blocat.

Întrebarea 50

Q:

Ce se poate întâmpla dacă funcția `f` este rulată de mai multe thread-uri simultane? De ce?

```
pthread_mutex_t m[2];
void* f(void* p) {
    int id = (int)p;
    pthread_mutex_t* first = &m[id % 2];
```

```

pthread_mutex_t* second = &m[(id+1) % 2];

pthread_mutex_lock(first);
pthread_mutex_lock(second);
...
pthread_mutex_unlock(second);
pthread_mutex_unlock(first);
}

```

A:

Răspuns:

Ar putea duce la deadlock. Dacă un thread are $id \% 2 = 0$ și celălalt $id \% 2 = 1$, și încearcă să blocheze simultan, primul ar bloca $m[0]$, celălalt ar bloca $m[1]$, iar amândoi vor aștepta pentru $m[1]$, respectiv $m[0]$ să se elibereze, simultan. Deci avem un deadlock.

Întrebarea 51

Q:

Dați un exemplu de valori distincte și mai mari ca 0 pentru T, N1, N2 și N3 pentru care programul de mai jos se încheie.

```

pthread_barrier_t b1, b2;

void* f1(void* a) {
pthread_barrier_wait(&b1) ;
return NULL;

void* f2(void* a) {
pthread_barrier_wait(&b2) ;
return NULL;

int main() {
int i;
pthread_t t[T][2];

pthread_barrier_init(&b1, NULL, N1);

pthread_barrier_init(&b2, NULL, N2);

for(i=0; i<T; i++) {
pthread_create(&t[i][0], NULL, f1, NULL);
pthread_create(&t[i][1], NULL, f2, NULL);

i

for(i=0; i<T; i++) {
pthread_join(t[i][0], NULL);
pthread_join(t[i][1], NULL);

y

```

```
pthread_barrier_destroy(&b1);  
pthread_barrier_destroy(&b2);  
return NULL;
```

A:

Pentru $T = 4$, $N1 = 8$ și $N2 = 4$ programul se încheie.

Explicație:

- $T = 4$ iterații create în bucla principală
- Fiecare iterație creează 2 thread-uri (unul pentru f1, unul pentru f2)
- Total: T thread-uri pentru f1 și T pentru f2
- Bariera b1 trebuie să aștepte T thread-uri = 4 (dacă vrei să treacă toate)
- Bariera b2 trebuie să aștepte T thread-uri = 4

DAR: dacă $N1 < T$, atunci doar primele $N1$ thread-uri trec bariera. Ca să se încheie, trebuie ca thread-urile blocate să poată trece — deci $N1$ trebuie să se divizeze cu numărul de thread-uri create. Valori care merg: $T = 4$, $N1 = 4$, $N2 = 4$ (sau $T = 6$, $N1 = 3$, $N2 = 2$ etc.).

Update Jonathan are dreptate: numărul barierei trebuie să fie un divizor al numărului de thread-uri.

Întrebarea 52

Q:

Ce puteți face ca programator pentru a preveni deadlock-urile?

A:

Deadlock-urile pot fi prevenite împiedicând cel puțin una dintre cele patru condiții necesare:

1. Eliminarea așteptării circulare: alegeți o ordine pentru resurse și blocați-le mereu în aceeași ordine.

2. Eliminarea excluderii mutuale:

- Resursele partajate cum ar fi fișierele read-only nu duc la deadlock, dar nu este posibil să eliminăm complet excluderea mutuală, deoarece unele resurse (cum ar fi imprimanta) sunt inherent non-partajabile.

3. Eliminarea condiției Hold and Wait:

- Alocăți toate resursele necesare procesului înainte de începerea execuției. Asta elimină condiția hold-and-wait dar duce la utilizare scăzută a dispozitivelor.

- Procesul cere resurse noi doar după ce le eliberează pe cele curente. Această soluție poate duce la starvation.

4. Eliminarea așteptării circulare:

- O modalitate este numerotarea tuturor resurselor și cerința ca procesele să le ceară doar în ordine strict crescătoare (sau descrescătoare).

5. Eliminarea condiției No Preemption:

- Preemptiunea alocării resurselor poate preveni această condiție când e posibil.

O metodă concretă pentru prevenirea deadlock-urilor este stabilirea unei ordini de blocare și respectarea ei mereu pentru a evita ciclurile. De exemplu, în problema filozofilor, putem bloca întotdeauna mai întâi furculița cu indicele mai mic, apoi pe cea cu indicele mai mare.

Întrebarea 53

Q:

Prin ce tranziție de stare va trece un process când apelează `sem_wait` și în ce condiții? Justificați răspunsul.

A:

Răspuns:

Dacă valoarea semaforului este 0, thread-ul va fi pus în WAIT până când un slot devine disponibil, pentru că altfel ar irosi un core fără să facă nimic.

Dacă valoarea este diferită de 0, atunci `sem_wait` nu va avea niciun efect asupra thread-ului respectiv, deci va rămâne probabil în RUN, dacă nu este comandat de alți factori externi să treacă în READY.

- dacă valoarea semaforului > 0 , procesul decrementează semaforul și continuă execuția în starea RUNNING

- dacă semaforul = 0, procesul trece în starea WAITING (blocat) până când semaforul este incrementat prin `sem_post`.

Întrebarea 54

Q:

Considerand ca dimensiunea unui bloc este B si dimensiunea unei adrese este A, cate blocuri de date sunt adresate de indirectarea dubla a unui i-nod?

A:

$(B/A)^2$ blocuri de date.

Explicație:

- Un bloc are dimensiunea B octeți
- O adresă (pointer la bloc) are dimensiunea A octeți
- Deci un bloc poate conține B/A adrese
- Indirectarea dublă: pointer → bloc cu B/A adrese → fiecare adresă spre un bloc cu B/A adrese → fiecare spre un bloc de date
- Total blocuri de date: $(B/A) \times (B/A) = (B/A)^2$

Întrebarea 55

Q:

Cand ati incarca in memorie paginile unui program care tocmai este pornit?

A:

Răspuns:

Aș încărca o pagină în memorie atunci când este cerută, deoarece dacă le încarc pe toate odată, efectul de paginare ar dispărea.

La pornire, aș încărca prima pagină, preîncărcând vecinii, bazat pe principiul localității. Acesta spune că un proces va avea nevoie în curând de paginile vecine paginii tocmai încărcate. După aceea, aș lua unul dintre vecini, repetând principiul.

Întrebarea 56

Q:

Dați trei expresii regulate care acceptă orice număr ne-negativ multiplu de 5.

A:

1. $^[0-9]^*[05]^\$$

2. $^0\$|^*[0-9]^*[05]^\$$

3. `^5$|^[0-9]*[05]$`

Cred ca ar mai merge si: `\<[0-9]*[05]\>` (nu stiu sigur daca trb sa fie toata linia sau doar numarul)

Întrebarea 57

Q:

Dați cinci comenzi GREP care afișează toate liniile dintr-un fișier care conțin litera "a" mare sau mic.

A:

`grep -i "a" filename`

`grep -Ei "a" filename`

`grep "[aA]" filename`

`grep -i "[a]+" filename`

`grep -iw "a" filename`

Observație: Pentru a treia, trebuie să fie `grep -E -i "[a]+" filename`. Fără -E încearcă să caute literalmente cuvântul `[a]+` și nu-l găsește.

Întrebarea 58

Q:

Scrieți două comenzi SED care afișează dintr-un fișier doar liniile care nu conțin cifra 7.

A:

`sed -E "/7/d" filename`

`sed -n "/7!/p" filename`

Mie ultima nu mi merge in terminal cand o scriu(imi da ceva eroare).

Poate se pune ca o comanda diferita asta: `sed -E "/[7]+/d" filename`

Saaaaau mai merge si asa: `sed -E "s/^(.*[7].*)$/gi" filename` (cu toate ca aici liniile care au 7 doar le face goale)

Întrebarea 59

Q:

Scrieți o comandă AWK care afișează suma penultimului câmp al tuturor liniilor.

A:

```
awk "{SUM += $(NF-1)} END{print SUM}" filename
```

Întrebarea 60

Q:

Cum puteți redirecta în linia de comanda ieșirea de eroare prin pipe înspre un alt program?

A:

Răspuns:

```
command 2>&1 | another_program
```

Sau:

- Redirecționează erorile unei comenzi într-un fișier: `rm fisier-inexistent.c 2> output.err`

- Redirecționează atât stdout cât și stderr în același fișier: `rm fisier-inexistent.c > output.all 2>&1` - citiți ca "redirecționează stdout în output.all, iar stderr în același loc unde merge stdout".

Întrebarea 61

Q:

Scrieți un script Shell UNIX care afișează toate argumentele din linia de comandă fără a folosi FOR.

A:

```
#!/bin/bash
echo "$@"
```

Sau, dacă vrem fiecare argument pe o linie separată:

```
#!/bin/bash
while [ $# -gt 0 ]; do
    echo "$1"
    shift
done
```

Explicație:

- "\$@" expandează la toate argumentele
 - shift mută argumentele la stânga (\$2 devine \$1, \$3 devine \$2, etc.)
 - \$# este numărul de argumente rămase
-

Întrebarea 62

Q:

Desenați ierarhia proceselor create de codul de mai jos, incluzând procesul părinte.

```
for(i=0; i<3; i++) {  
    fork();  
    execlp("ls", "ls", "/", NULL);  
}
```

A:

```
for(i=0; i<3; i++) {  
  
    fork();  
  
    execlp("ls", "ls", "/", NULL);  
  
}
```

La iterația 0: P face fork → apar P și C1. Ambele intră în execlp și devin ls.

Dacă execlp reușește, NU se continuă bucla.

Rezultat (cu exec reușit):

P → C1

Doar 1 proces copil este creat.

(Notă: codul original sugerează 3 copii – asta presupune că exec eșuează și bucla continuă.

În practică ls există, deci doar 1 copil e creat.)

Întrebarea 63

Q:

Adăugați codul C necesar pentru ca fișierul b.txt să fie suprascris cu conținutul fișierului a.txt din instrucțiunea de mai jos.

```
execlp("cat", "cat", "a.txt", NULL);
```

A:

Răspunsul anterior era incomplet (lipsea O_TRUNC | O_CREAT, dup2 avea typo).

Corect:

```
int fd = open("b.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);  
dup2(fd, STDOUT_FILENO); // STDOUT_FILENO = 1  
close(fd);  
execlp("cat", "cat", "a.txt", NULL);
```

Flag-urile sunt esențiale:

- O_CREAT: creează b.txt dacă nu există
- O_TRUNC: șterge conținutul vechi (suprascriere)
- O_WRONLY: deschide doar pentru scriere

Întrebarea 64

Q:

De ce nu e recomandat sa comunicați bidirecțional printr-un singur FIFO?

A:

Trebuie să creăm 2 FIFO-uri pentru comunicarea bidirecțională, deoarece un FIFO poate transmite date doar într-o singură direcție și astfel evităm comportament neașteptat al programului.

Folosirea unui singur FIFO poate duce la race conditions și deadlock-uri.

În plus: nu suntem siguri de ordinea în care procesele vor scrie în FIFO, ceea ce poate cauza corupere de date. Să presupunem că procesul A vrea să scrie "abc" și apoi procesul B "123". Dacă A pierde CPU-ul în timpul scrierii, putem avea ceva de genul "a1bc23".

Întrebarea 65

Q:

Câte FIFO-uri poate deschide un process dacă nu sunt și nici nu vor fi folosite vreodată de vreun alt proces?

A:

Numărul maxim de FIFO-uri pe care le poate deschide un proces este limitat doar de numărul maxim de file descriptors al sistemului (verificabil cu 'ulimit -n', tipic 1024).

ATENȚIE la blocare: open() pe un FIFO se blochează dacă nu există proces complementar:

- open(FIFO, O_RDONLY) așteaptă să existe un proces care îl deschide cu O_WRONLY
- open(FIFO, O_WRONLY) așteaptă să existe un proces care îl deschide cu O_RDONLY

Dacă FIFO-ul nu este folosit de nimeni altcineva, primul open() blochează la nesfârșit.

Excepție: open cu flag-ul O_NONBLOCK nu se blochează (dar poate eșua).

Întrebarea 66

Q:

Când ați folosi un process în locul unui thread?

A:

- Procesele au propriul spațiu de memorie, oferind izolare și reducând coruperea memoriei.
- Procesele pot fi distribuite pe mai multe mașini, în timp ce thread-urile sunt limitate la resursele unei singure mașini.
- Procesele oferă granițe de securitate mai bune, deoarece nu partajează același spațiu de adrese.
- O eroare într-un proces nu afectează celelalte procese.

Întrebarea 67

Q:

De ce un thread trebuie să reverifice condiția la ieșirea din apelul `pthread_cond_wait`?

`pthread_cond_wait` call?

A:

Un thread trebuie să reverifice condiția la întoarcerea din `pthread_cond_wait` din 2 motive:

1. SPURIOUS WAKEUPS: standardul POSIX permite ca `pthread_cond_wait` să se trezească uneori FĂRĂ ca cineva să fi apelat `pthread_cond_signal`. Asta se cheamă spurious wakeup. Deci thread-ul trebuie să verifice că condiția e cu adevărat îndeplinită.
2. ALTE THREAD-URI: alte thread-uri pot fi trezite înaintea celui curent și pot consuma resursa cerută, lăsând condiția invalidă din nou până ajunge thread-ul nostru.

De aceea, modelul corect este BUCLĂ WHILE (nu if):

```
while (!condition) {  
    pthread_cond_wait(&cond, &mutex);  
}
```

Întrebarea 68

Q:

Care va fi efectul înlocuirii apelurilor la `pthread_mutex_lock` cu apeluri la `pthread_rwlock_rdlock`?

`pthread_mutex_lock` with calls to `pthread_rwlock_rdlock`?

A:

Va cauza probleme deoarece `pthread_rwlock_rdlock` este folosit pentru blocaje de citire, permițând mai multor thread-uri să acceseze secțiunea critică, ceea ce nu asigură excluderea mutuală și ar putea duce la race conditions.

Întrebarea 69

Q:

Care e efectul apelului la `pthread_barrier_wait` pentru o barieră inițializată cu 1?

A:

Va elibera imediat thread-ul care așteaptă, deoarece condiția barierei este deja satisfăcută. Nu va avea nevoie de nicio așteptare, iar thread-ul va continua.

(Va funcționa similar cu un `pthread_mutex`.)

Întrebarea 70

Q:

Cum puteți incrementa valoarea unui semafor?

A:

Folosind funcția `sem_post()`:

```
#include <semaphore.h>
sem_post(&sem);
```

`sem_post` incrementează valoarea semaforului cu 1. Dacă există thread-uri care așteaptă pe `sem_wait`, unul dintre ele este trezit.

Este operația V (sau "up", sau "signal") din terminologia clasică a semaforilor.

Întrebarea 71

Q:

Ce puteți face ca programator pentru a preveni deadlock-urile?

A:

- Folosiți o ordine consistentă de blocare
 - Evitați blocaje imbricate
 - Utilizați ierarhii de resurse
-

Întrebarea 72

Q:

Prin ce tranziție de stare va trece un process când citește dintr-un fișier?

A:

Când un proces citește dintr-un fișier, trece din starea de execuție (running) în starea blocată (blocked/wait) dacă datele nu sunt imediat disponibile, și revine în ready și apoi running odată ce datele sunt disponibile.

Întrebarea 73

Q:

Ce conține superblocul unui disc Linux?

A:

filesystem type(ext4, ext3), size information(blocks, free blocks, size), state information, inode information, uuid, mount information

In curs scrie:

Blocul 1 este numit și superbloc. In el sunt trecute o serie de informații prin care se definește

sistemul de fișiere de pe disc. Printre aceste informații amintim:

- numărul n de inoduri (detaliem imediat);
 - numărul de zone definite pe disc;
 - pointeri spre harta de biți a alocării inodurilor;
 - pointeri spre harta de biți a spațiului liber disc;
 - dimensiunile zonelor disc, etc.
-

Întrebarea 74

Q:

Se poate crea un link hard spre un fișier aflat pe o altă partiție? Justificați răspunsul.

A:

Nu, deoarece hard link-urile trebuie să fie pe același sistem de fișiere, deoarece referențiază i-node-ul, iar i-node-urile sunt unice doar în cadrul sistemului lor de fișiere.

Întrebarea 75

Q:

Dați o expresie regulată care acceptă orice număr impar de cuvinte separate prin spații, fiecare cuvânt având număr impar de litere.

A:

```
^([a-z]{2}*[a-z] [a-z]{2}+ ){0,}([a-z]{2}*[a-z] [a-z]{2}+)*$ -asta a mers pe terminal
```

Solutia mea (merge pe terminal):

```
grep -E -i "^(([a-z]){2}*[a-z] ){2}*([a-z]){2}*[a-z]$" a.txt
```

```
<\(..)*.\> (<\(..)*.\> <\(..)*.\>)*
```

Întrebarea 76

Q:

Dați patru comenzi care afișează numărul de linii goale dintr-un fișier.-8, 8

A:

```
grep -c "^$" filename
```

```
grep -Ec "^$" filename
```

```
awk "BEGIN {count = 0} /^$/ {count++} END {print count}" filename
```

```
sed -n "/^$/p" filename | wc -l
```

Merge si asta:

```
grep -E -v -c "." filename
```

```
grep -E -c "^$" a.txt
```

```
grep -E -V -c "..*" a.txt
```

```
grep -E -v -c ".*" a.txt
```

```
grep -E -v -c ".{1,}" a.txt
```

Întrebarea 77

Q:

Scrieți o comandă SED care afișează liniile dintr-un fișier ștergând din ele primul, al treilea, al cincilea, al șaptelea, etc spații.

A:

Comandă SED care șterge primul, al 3-lea, al 5-lea, etc. spațiu de pe fiecare linie:

```
sed -E ':a; s/^( [^ ]*) ([^ ]*)( |$)( [^ ]*) /\1\2\3\4/; ta' fisier
```

Sau o variantă mai simplă: înlocuiește perechi de spații cu un singur spațiu, începând de la primul:

```
sed -E 's/([ ^ ]+) ([^ ]+) /\1\2 /g' fisier
```

Explicație: se șterge spațiul din poziție impară (între cuvintele 1-2, 3-4, 5-6, ...).

Întrebarea 78

Q:

Scrieți o comandă AWK care afișează produsul ultimului câmp al liniilor de pe poziții impare care au număr impar de câmpuri.

A:

```
awk 'BEGIN {p=1} NR%2==1 && NF%2==1 {p *= $NF} END {print p}' fisier
```

Explicație:

- BEGIN {p=1} = inițializează p = 1 (produs inițial)
- NR%2==1 = doar liniile de pe poziție impară (NR = Number of Record)
- NF%2==1 = doar liniile cu număr impar de câmpuri (NF = Number of Fields)
- {p *= \$NF} = înmulțește p cu ultimul câmp (\$NF = last field)
- END {print p} = la final afișează produsul

Atenție: folosiți apostrof simplu (') în jurul programului AWK, nu ghilimele duble – altfel shell-ul va încerca să interpreteze \$NF.

Întrebarea 79

Q:

Dați patru moduri prin care ieșirea standard a unui proces poate fi redirectată.

A:

```
command > file
```

```
command >> file
```

command | file

command > file 2>&1

Asta nu mi merge pe terminal:

command | file (ok, am revenit si cred ca merge, doar trebuia sa o scriu altfel:)))

Poate asta: (?)

command 2> a.txt

Nu stiu, mai merge intreat pe cineva care stie:))

Redirect it via > : command > a.txt

Redirect it via >> : command >> a.txt

Redirect it via pipe : command | echo > a.txt

Redirect it with tee: echo "Hello" | tee greetings.txt

Întrebarea 80

Q:

Scrieți trei condiții Shell UNIX care verifică existența unui fișier.

A:

```
if test -e fisier; then
```

```
    echo "Există"
```

```
fi
```

```
if [ -e fisier ]; then
```

```
    echo "Există"
```

```
fi
```

```
if [[ -e fisier ]]; then
```

```
    echo "Există"
```

```
fi
```

Alte variante:

```
1) if [ -e $fisier ]
```

```
2) if [ $(ls $fisier 2>&1 | grep -E -c "No such file") -eq 0 ]
```

```
3) if $(ls $fisier 2>&1 | grep -E -q "No such file")
```

Întrebarea 81

Q:

Adăugați codul C necesar pentru ca instrucțiunea de mai jos să nu se blocheze așteptând la intrarea standard.

```
execlp("cat", NULL);
```

A:

Cum NU se blochează cat așteptând la stdin? Îi dăm un fișier ca stdin folosind dup2:

```
int fd = open("a.txt", O_RDONLY);
```

```
dup2(fd, STDIN_FILENO); // STDIN_FILENO = 0, stdin merge acum din a.txt
```

```
close(fd);
```

```
execlp("cat", "cat", NULL);
```

ATENȚIE: răspunsul original cu "O_WRONLY" e GREȘIT. Pentru citire folosim O_RDONLY.

STD_IN nu există ca constantă – corect e STDIN_FILENO (sau 0).

Întrebarea 82

Q:

Schițați o implementare a funcțiilor popen și pclose, doar pentru cazul în care outputul comenzii e citit în codul C.

A:

popen:

- creează un pipe

- face fork procesului

- copil: închide capătul de citire al pipe-ului, redirectează stdout către pipe și execută comanda cu execlp

- părinte: închide capătul de scriere al pipe-ului, returnează un pointer FILE* pentru citire

pclose:

- închide pointer-ul de fișier
 - așteaptă procesul copil
 - returnează statusul de ieșire al copilului
-

Întrebarea 83

Q:

Câte FIFO-uri poate deschide pentru citire un process, dacă FIFO-urile sunt și vor fi întotdeauna folosite de alte procese doar pentru citire?

A:

Depinde. Comportamentul special al FIFO este că așteaptă să fie deschis pentru operația complementară. Dacă fiecare dintre celelalte procese deschide exact un FIFO, atunci putem deschide câte vrem. Altfel, dacă nu suntem atenți, poate duce la deadlock.

Să presupunem că avem FIFO-urile 1 și 2. Fie A un proces care deschide FIFO-urile în ordinea 1, 2. Dacă celălalt proces deschide FIFO-urile în ordinea 2, 1, avem deadlock și nu mai putem avansa.

Depinde și de limita descriptorilor de fișier a sistemului, care poate fi verificată cu "ulimit -n".

Întrebarea 84

Q:

Când ați folosi un FIFO în locul unui pipe?

A:

Aș folosi un FIFO în loc de pipe atunci când trebuie să lucrez cu procese care provin din programe diferite și care nu au un strămoș comun. Pentru ca cele două să poată comunica, ar avea nevoie de un fișier pe disc, ceva ce un pipe nu poate oferi, așa că FIFO-ul ar fi soluția.

Diferențele dintre FIFO și pipe:

- FIFO-urile sunt fișiere pe disc, deci au un ID unic la nivel de sistem (calea fișierului) pe care procesele îl pot folosi pentru a le adresa.
- Crearea pipe-ului îl și deschide, dar FIFO-urile trebuie create și deschise explicit.

- Pipe-urile sunt distruse când toate capetele sunt închise; FIFO-urile trebuie șterse explicit.
 - FIFO-urile pot trăi dincolo de procesele care le folosesc.
-

Întrebarea 85

Q:

Ce este o "secțiune critică"?

A:

O secțiune critică este o parte a programului care accesează resurse partajate și le modifică într-un mod în care modificările nu sunt efectuate în operații atomice. Pentru această secțiune, resursele partajate trebuie protejate pentru a evita accesul concurrent. Această secțiune protejată nu poate fi accesată de mai mult de un proces/thread o dată.

O secțiune critică conține operații care nu sunt atomice, deci trebuie protejată folosind un mecanism de sincronizare cum ar fi mutex-uri, read-write locks, semafoare etc.

Întrebarea 86

Q:

Când ați folosi un mutex în locul unui rwlock?

A:

- Când un program are mai multe operații de scriere decât de citire, mutex-ul ar fi mai eficient.
- Simplitate.
- Evitarea deadlock-urilor: mutex-urile sunt mai puțin predispuse la deadlock-uri.

Aș folosi un mutex în loc de rwlock când lucrez cu variabile condiționale (pthread_cond_wait poate elibera doar mutex-uri, nu rwlock-uri). De asemenea, read locks sunt utile doar când un thread efectuează operații exclusiv de citire; dacă toate thread-urile fac operații neatomice, un rwlock nu se justifică.

Întrebarea 87

Q:

Care va fi efectul înlocuirii apelurilor la pthread_mutex_lock cu apeluri la sem_wait?

pthread_mutex_lock with calls to sem_wait?

A:

`sem_wait` decrementează semaforul și poate permite mai multor thread-uri să intre în secțiunea critică, ceea ce ar duce la race conditions și corupere de date.

Trei cazuri:

1) Înlocuim doar `sem_wait` cu `mutex_lock`, dar nu și `mutex_unlock` cu `sem_post` → comportament nedefinit cauzat de deblocarea unui mutex care nu a fost blocat.

2) Folosim un semafor binar → la fel ca un mutex; la fiecare trecere semaforul este decrementat la 0, iar la `sem_post` este incrementat.

3) Folosim un semafor generic cu x thread-uri alocate → thread-urile trec până când semaforul permite, iar în funcție de operațiile executate rezultatele pot fi imprevizibile.

Întrebarea 88

Q:

Ce face `pthread_cond_wait` cu mutex-ul primit ca argument?

`pthread_cond_wait` do with the mutex it gets as argument?

A:

- Eliberează mutex-ul, permițând altor thread-uri să-l obțină.
- Pune thread-ul "la somn" și așteaptă să fie semnalizată variabila condițională.
- Reachiziționează mutex-ul, asigurând acces exclusiv la resursele partajate.

`pthread_cond_wait` deblochează mutex-ul când este apelată. Când e trimis un semnal (prin signal sau broadcast), `pthread_cond_wait` așteaptă ca mutex-ul să fie liber, îl achiziționează și îl blochează, apoi continuă execuția programului.

Întrebarea 89

Q:

Schițați o soluție pentru problema producător-consumator.

A:

Răspuns:

N = dimensiunea buffer-ului

p = următorul index pentru inserare obiect

c = următorul index pentru extragere obiect

Consumator: ia obiecte cât timp nu e buffer-ul gol ($p == c$).

Producător: produce obiecte cât timp nu e buffer-ul plin ($(p+1) \% N == c$).

Folosim 2 semafoare:

- unul cu valoare inițială 0 (indică sloturile ocupate)
 - unul cu valoare inițială N (indică sloturile libere)
 - plus un mutex (sau semafor binar) pentru excluderea mutuală la modificarea buffer-ului.
-

Întrebarea 90

Q:

Ce puteți face ca programator pentru a preveni deadlock-urile?

A:

- Folosiți o ordine consistentă de blocare.
 - Evitați blocaje imbricate.
 - Folosiți algoritmi de detectare a deadlock-urilor.
 - Folosiți timeout-uri pentru blocare.
-

Întrebarea 91

Q:

Prin ce tranziție de stare va trece un process când apelează `pthread_cond_wait`? Justificați răspunsul.

`pthread_cond_wait`? Justify your answer.

A:

- Când `pthread_cond_wait` este apelată, thread-ul eliberează mutex-ul asociat și se blochează.

- Rămâne blocat până când un alt thread semnalizează condiția cu `pthread_cond_signal` sau `pthread_cond_broadcast`.
- După ce condiția este semnalizată, thread-ul este trezit, reacheziționează mutex-ul și revine în starea `RUNNING`.

Un semafor este un întreg a cărui valoare nu poate scădea sub zero. Două operații pot fi efectuate pe semafoare: incrementarea (`sem_post`) și decrementarea (`sem_wait`). Dacă valoarea unui semafor este 0, `sem_wait` va bloca thread-ul până când valoarea devine mai mare decât 0.

Când un proces apelează `pthread_cond_wait`, va trece din `RUN` în `WAIT`. Acest comportament este cauzat de faptul că, din moment ce thread-ul va aștepta un semnal de la alt thread, procesorul ar fi ocupat fără motiv, așa că eliberează core-ul pentru ca alt proces să-și completeze taskurile.

Întrebarea 92

Q:

Ce conține un fișier de tip director în sistemul de fișiere Linux?

A:

Intrări de director (nume de fișiere, numere de i-node):

- "." (punct) reprezintă directorul însuși, are i-node-ul directorului
- ".." (două puncte) reprezintă directorul părinte, i-node-ul părintelui
- intrări obișnuite: nume de fișier sau subdirector și i-node-ul specific.

Întrebarea 93

Q:

Explicați diferența dintre un link simbolic și un link hard.

A:

Link simbolic (soft link):

- indică spre calea fișierului țintă
- poate traversa sisteme de fișiere diferite
- devine invalid dacă fișierul țintă este șters

- creat cu "ln -s"

Hard link:

- indică spre i-node-ul fișierului țintă
- nu poate traversa sisteme de fișiere diferite
- rămâne valid atâta timp cât există cel puțin un link
- creat cu "ln"

Întrebarea 94

Q:

Scrieți o expresie regulată care acceptă orice linii care conțin cel puțin trei vocale.

A:

`[aeiouAEIOU].*[aeiouAEIOU].*[aeiouAEIOU]`

`^(.*[aeiouAEIOU]){3}.*$`

Întrebarea 95

Q:

Care este principiul localității cu privire la încărcarea paginilor?

A:

Un proces va avea nevoie probabil în curând de paginile vecine paginii tocmai încărcate. Prezici de ce ar avea nevoie programul și, când încarci o pagină, preîncarci și vecinii ei.

Când încărcăm o pagină, încărcăm și paginile din jurul ei, pentru că probabil vom avea nevoie de ele în curând.

Întrebarea 96

Q:

Planificați joburile de mai jos (Nume/Durată/Termen-limită) astfel încât suma întârzierilor să fie minimizată.

A/5/7 B/2/4 C/4/13 D/3/8

A:

Ordinea optimă (algoritm SJF cu termen-limită): B, A, D, C

B/2/4: durată 2, termen 4

A/5/7: durată 5, termen 7

D/3/8: durată 3, termen 8

C/4/13: durată 4, termen 13

Calculul întârzierilor:

B: pornește 0, termină 2 → întârziere 0 (termen 4)

A: pornește 2, termină 7 → întârziere 0 (termen 7)

D: pornește 7, termină 10 → întârziere 2 (termen 8)

C: pornește 10, termină 14 → întârziere 1 (termen 13)

Întârziere totală = 3.

Întrebarea 97

Q:

Care este riscul adus de funcția `f` când e executată în mai multe thread-uri simultane? Justificați răspunsul.

```
pthread_mutex_t m[2];
void* f(void* p) {
    int id = (int)p;
    pthread_mutex_t* first = &m[id % 2];
    pthread_mutex_t* second = &m[(id+1) % 2];

    pthread_mutex_lock(first);
    pthread_mutex_lock(second);
    ...
    pthread_mutex_unlock(second);
    pthread_mutex_unlock(first);
}
```

A:

Există risc de deadlock, deoarece thread-urile nu blochează cele 2 mutex-uri în aceeași ordine.

De exemplu, dacă există 2 thread-uri cu id-urile 0 și 1:

Thread 0: `pthread_mutex_lock(&m[0]);`

Thread 1: `pthread_mutex_lock(&m[1]);`

Să presupunem că amândoi reușesc să achiziționeze mutex-ul. Apoi:

Thread 0 are mutex 0, Thread 1 are mutex 1.

La pasul următor, ar încerca să execute:

```
Thread 0: pthread_mutex_lock(&m[1]);
```

```
Thread 1: pthread_mutex_lock(&m[0]);
```

Deoarece Thread 1 are mutex 1, Thread 0 ar aștepta. Deoarece Thread 0 are mutex 0, Thread 1 ar aștepta. Astfel, ambele așteaptă unul după altul → deadlock.

Întrebarea 98

Q:

Dați un avantaj și un dezavantaj al politicii de plasare First Fit față de politica Worst Fit.

A:

Avantaj: First Fit este mai rapid decât Worst Fit, deoarece nu trebuie să parcurgă toată lista de blocuri libere.

Dezavantaj: Are mai multe șanse să lase mici găuri în memorie => fragmentare a memoriei.

Întrebarea 99

Q:

Considerând că dimensiunea unui bloc este B și dimensiunea unei adrese este A, câte blocuri de date sunt adresate de indirectarea triplă a unui i-node?

A:

$$(B/A) * (B/A) * (B/A) = B^3/A^3$$

De ce? Pentru că avem 3 niveluri de indirectare, iar fiecare nivel este un bloc de dimensiune B/A. Astfel, avem B/A blocuri în primul nivel, B/A blocuri în al doilea nivel și B/A blocuri în al treilea nivel.

Întrebarea 100

Q:

Scrieți o expresie regulată care acceptă linii care conțin "a", dar nu "b".

A:

$^{\wedge}[^b]^*a[^b]^*\$$

Sau folosind lookahead negativ:

`^(?!.*b).*a.*$`

Explicație:

`(?!.*b)` => lookahead negativ, înseamnă că expresia nu se va potrivi dacă linia conține "b"

`.a.*` => se potrivește cu orice linie care conține "a"

`.*$` => se potrivește cu sfârșitul liniei

Variantă cu grep:

`grep "a" file_name | grep -v "b"`

Întrebarea 101

Q:

Care este numărul maxim de procese copil, create de codul de mai jos, care pot coexista simultan?

```
for(i=0; i<7; i++) {  
    if (fork() == 0) {  
        sleep(rand() % 10);  
        exit(0);  
    }  
    if(i % 3 == 0) {  
        wait(0);  
    }  
}
```

A:

5

$i = 0 \Rightarrow$ Se creează C1 $\Rightarrow$ 1 proces copil; așteaptă să se termine $\Rightarrow$ 0 copii

$i = 1 \Rightarrow$ Se creează C2, nu așteaptă $\Rightarrow$ 1 copil

$i = 2 \Rightarrow$ Se creează C3, nu așteaptă $\Rightarrow$ 2 copii

$i = 3 \Rightarrow$ Se creează C4 $\Rightarrow$ 3 copii; așteaptă să se termine $\Rightarrow$ 2 copii

$i = 4 \Rightarrow$ Se creează C5 $\Rightarrow$ 3 copii

$i = 5 \Rightarrow$ Se creează C6 $\Rightarrow$ 4 copii

$i = 6 \Rightarrow$ Se creează C7 $\Rightarrow$ 5 copii; așteaptă $\Rightarrow$ 4 copii

Maxim simultan: 5 copii.

Întrebarea 102

Q:

Câte thread-uri ați folosi pentru procesarea unui milion de fișiere? Justificați răspunsul.

A:

Să presupunem că avem k core-uri (nuclee). Atunci putem folosi k thread-uri pentru procesarea fișierelor. Asta deoarece numărul de thread-uri ar trebui să fie egal cu numărul de core-uri, pentru a folosi toate core-urile în același timp.

Dacă folosim mai multe thread-uri decât core-uri, thread-urile vor trebui să aștepte unele după altele (context switch frecvent), iar asta nu va fi eficient.

Dacă folosim mai puține thread-uri decât core-uri, nu vom folosi toate core-urile disponibile.

Întrebarea 103

Q:

De ce o operație I/O cauzează un proces să treacă din starea running în starea waiting?

A:

Deoarece operațiile I/O pot dura mult, planificatorul (scheduler) pune procesul în starea Waiting și rulează alt proces care e în starea Ready. Acest lucru permite o utilizare mai eficientă a procesorului — în loc să stea inactiv așteptând I/O, CPU-ul lucrează pentru un alt proces. (Scheduler-ul este preemptiv, anticipativ.)

Întrebarea 104

Q:

Cum se face calculul adresei în alocarea cu partiții fixe absolute?

A:

Adresa este aceeași cu adresa fizică. Sistemul de operare nu face nicio translatare.

În alocarea cu partiții fixe absolute, există un număr predefinit de partiții în RAM care pot fi folosite de un program. Acest număr de partiții este fix, iar un program este compilat pentru o anumită partiție.

Aceasta înseamnă că adresa finală executabilă a unei variabile în RAM este cunoscută la compilare, deci adresa din binar este aceeași cu adresa finală în executabil:

Adresa binară = Adresa fizică.

La compilare, programul folosește adresa de început a partiției pentru care este compilat ca să determine adresa finală exactă.

Întrebarea 105

Q:

Procesele A, B și C comunică prin FIFO-urile X, Y și Z conform diagramei de mai jos. Schițați fragmentele de cod care deschid FIFO-urile în cele trei procese.

A --X--> B

B --Y--> C

C --Z--> A

A:

```
//A.cpp
```

```
int a2b = open("X", O_WRONLY);
```

```
int c2a = open("Z", O_RDONLY);
```

```
//B.cpp
```

```
int a2b = open("X", O_RDONLY);
```

```
int b2c = open("Y", O_WRONLY);
```

```
//C.cpp
```

```
int b2c = open("Y", O_RDONLY);
```

```
int c2a = open("Z", O_WRONLY);
```

InA

Open de write pt X

Open de read pt Z

Inb

Open de write pt Y, si ree pt X

InC open de read pt Y, write pt Z si

Întrebarea 106

Q:

15)

Dati un exemplu de valori pentru T, N1, N2 si N3 pentru care programul de mai jos se incheie

```
pthread_barrier_t b1, b2, b3;
```

```
void* F1(void* a) {  
pthread_barrier_wait(&b1);  
return NULL;
```

```
void* f2 (void a) {  
pthread_barrier_wait(&b2);  
return NULL;
```

»

```
void* f2 (void a) {  
pthread_barrier_wait(&b3);  
return NULL;
```

?

```
int main() {  
int 4;  
pthread_t t[T][3];  
pthread_barrier_init(&b1, N1);  
pthread_barrier_init(&b2, N2);  
pthread_barrier_init(&b3, N3);
```

```
for (1-05 i<Ts is) {  
pthread_create(&t[i][0], NULL, #1,  
pthread_create(&t[i][1], NULL, f2,  
pthread_create(&t[i][2], NULL, f3,  
?
```

```
for (i=05 i<Ts t+) f  
pthread_join(t[i][0], NULL);  
pthread_join(t[i][1], NULL);  
pthread_join(t[i][2], NULL);
```

x

```
pthread_barrier_destroy(&b1);  
pthread_barrier_destroy(&b2);  
pthread_barrier_destroy (&b3);
```

```
return NULL;
```

```
NULL);  
NULL);  
NULL);
```

A:

Cod original (cu OCR-uri): trei bariere b1, b2, b3, fiecare așteaptă N1, N2, N3 thread-uri.

Bucula creează T iterații $\times 3$ thread-uri = $3T$ thread-uri.

Pentru ca programul să se încheie:

- Bariera $b1$ are nevoie de T thread-uri (sau T să fie multiplu de $N1$)
- Bariera $b2$ are nevoie de T thread-uri (sau T să fie multiplu de $N2$)
- Bariera $b3$ are nevoie de T thread-uri (sau T să fie multiplu de $N3$)

Valoare simplă: $T = N1 = N2 = N3 = 3$ (toate identice, toate trec).

Alternativă: $T = 6, N1 = 2, N2 = 3, N3 = 6$ (T trebuie să fie divizibil cu Ni).

Întrebarea 107

Q:

Câte procese va crea codul de mai jos, excluzând procesul părinte?

```
for(i=0; i<8; i++) {  
    if(fork() && i % 2)  
        break;  
}
```

A:

```
for(i=0; i<8; i++) {  
    if(fork() && i % 2)  
        break;  
}
```

Logica: `fork()` returnează PID-ul copilului în PĂRINTE (truthy) și 0 în COPIL (falsy).

$\&\& i\%2 \rightarrow$ părintele iese din buclă doar pe iterații IMPARE ($i \% 2 = 1$).

$i = 0$: P fork $\rightarrow$ C1. P continuă ($i\%2=0$). C1 continuă.

$i = 1$: P fork $\rightarrow$ C2. P break ($i\%2=1$). C1 fork $\rightarrow$ C3. C1 break.

$i = 2$: C2, C3 continuă. C2 fork $\rightarrow$ C4. C3 fork $\rightarrow$ C5.

$i = 3$: C2, C3, C4, C5 continuă. Toți fork. break la fiecare $\rightarrow$ C6, C7, C8, C9.

Total copii creați: 9.

Întrebarea 108

Q:

Ce va afișa în consolă fragmentul de cod de mai jos, considerând că thread-urile se creează fără probleme? Justificați răspunsul.

```
/* Considerați că header-ele necesare sunt incluse aici */
void* f(void* a) {
    printf("%d\n", *(int*)a);
    return NULL;
}

int main() {
    /* Considerați că variabilele necesare sunt declarate aici */
    for(i=0; i<10; i++) {
        pthread_create(&t[i], NULL, f, &i);
    }
    /* Considerați că aici se fac join-urile necesare */
    return 0;
}
```

A:

Nu este cert ce vor afișa thread-urile. Output-ul va fi alcătuit din 10 numere, fiecare între 0 și 10, dar ordinea și valorile exacte nu sunt certe.

Aceasta deoarece thread-urile primesc adresa lui *i* ca parametru, dar *i* este incrementat în timpul buclei for. Astfel, nu putem determina ce se întâmplă: poate *i* este incrementat de mai multe ori înainte să fie afișat de un thread, sau poate mai multe thread-uri afișează același *i*. Poate un thread care a citit valoarea lui *i* într-un moment așteaptă, iar alt thread ajunge să afișeze un *i* mai mare înainte ca thread-ul curent să poată afișa valoarea.

Întrebarea 109

Q:

Planificați joburile de mai jos (date ca Nume/Durată/Termen-limită) astfel încât suma întârzierilor să fie minimizată.

A/3/8, B/10/15, C/3/5

A:

Ordinea: CAB (întârziere totală = 1 secundă)

C: pornește la 0, termină la 3, întârziere = 0 (termen 5)

A: pornește la 3, termină la 6, întârziere = 0 (termen 8)

B: pornește la 6, termină la 16, întârziere = 1 (termen 15)

Întârziere totală = 1.

Întrebarea 110

Q:

Date două cache-uri set-asociative, una cu 2 seturi de 4 pagini și alta cu 4 seturi de 2 pagini, care ar performa mai bine pentru secvența de cereri de pagini de mai jos? Justificați răspunsul.

17, 2, 37, 6, 9

A:

Pentru cache-uri set-asociative, ca să plasăm o pagină în cache, calculăm mai întâi numărul paginii modulo numărul de seturi, apoi căutăm un loc liber în acel set.

Prima variantă: 2 seturi de 4 pagini.

$17 \bmod 2 = 1 \rightarrow$ în setul 1

$2 \bmod 2 = 0 \rightarrow$ în setul 0

$37 \bmod 2 = 1 \rightarrow$ în setul 1

$6 \bmod 2 = 0 \rightarrow$ în setul 0

$9 \bmod 2 = 1 \rightarrow$ în setul 1

A doua variantă: 4 seturi de 2 pagini.

$17 \bmod 4 = 1$

$2 \bmod 4 = 2$

$37 \bmod 4 = 1$

$6 \bmod 4 = 2$

$9 \bmod 4 = 1 \rightarrow$ ambele pagini din acest grup sunt pline, cache-ul trebuie să elimine o pagină.

Prima variantă (2 seturi de 4 pagini) ar performa mai bine pentru această secvență, deoarece nu trebuie să dea afară o pagină din cache pentru a încărca alta din RAM.

Întrebarea 111

Q:

Considerând că un bloc poate conține N adrese către alte blocuri, câte blocuri de date sunt adresate de indirectările dublă și triplă ale unui i-node împreună?

$N*N + N*N*N$

A:

De ce? Pentru că indirectarea dublă indică spre N blocuri, fiecare indicând spre alte N blocuri (deci $N*N$), iar indirectarea triplă indică spre N blocuri, fiecare indicând spre N blocuri, fiecare indicând spre alte N blocuri (deci $N*N*N$).

Întrebarea 112

Q:

Dați un avantaj și un dezavantaj al alocării cu partiții fixe față de alocarea cu partiții relocatabile.

A:

Avantaj: Calculul adreselor este mai ușor, deoarece știm unde se află fiecare proces în memorie.

Dezavantaj: Nu putem folosi memoria eficient, deoarece s-ar putea să rămână spațiu nefolosit dacă un proces nu încapă într-o partiție, ducând la fragmentare a memoriei.

Întrebarea 113

Q:

Dați un avantaj și un dezavantaj al încărcării tuturor paginilor unui proces în memorie odată, comparativ cu încărcarea la cerere.

A:

Avantaj: Când încercăm să accesăm o pagină, ea este deja în memorie, deci nu trebuie să așteptăm să fie încărcată.

Dezavantaj: S-ar putea să încărcăm pagini de care nu avem nevoie, irosind memorie. De asemenea, durează mai mult să încarci toate paginile odată.

Întrebarea 114

Q:

Dați un avantaj și un dezavantaj al cache-ului cu acces direct comparativ cu cache-ul asociativ.

A:

Avantaj: Este mai rapid să cauți o pagină în cache, deoarece știm exact unde să o căutăm.

Dezavantaj: Nu putem avea mai multe pagini cu același index în cache, deci s-ar putea să fim forțați să eliminăm o pagină chiar dacă mai e necesară (cache thrashing).

Întrebarea 115

Q:

Când își schimbă un proces starea din WAIT în RUN?

A:

Un proces își schimbă starea din WAIT în RUN (sau, mai exact, mai întâi în READY) când evenimentul sau condiția pe care o aștepta se întâmplă sau devine disponibilă. Procesul trece dintr-o stare blocată/de așteptare într-o stare gata-de-rulare când devine eligibil pentru a fi planificat la execuție de către sistemul de operare.

Întrebarea 116

Q:

De ce se poate crea un hard-link către fișiere de pe aceeași partiție, dar nu către fișiere de pe partiții diferite?

A:

Un hard-link indică spre același i-node ca și fișierul. Deoarece i-node-ul este o structură de date folosită pentru a reprezenta un obiect al sistemului de fișiere, este intern sistemului de fișiere și nu poți indica spre un i-node al altui sistem de fișiere.

Fiecare sistem de fișiere are propria organizare și structuri de date pentru gestionarea fișierelor și directoarelor (superblock, tabel de i-noduri, blocuri de date). Diferite partiții pot avea tipuri și configurări diferite de sisteme de fișiere.

Soft link-urile (linkuri simbolice) pot traversa sistemele de fișiere, deoarece doar indică spre o altă intrare de director (interfața sistemului de fișiere, nu un obiect intern).

Întrebarea 117

Q:

Adăugați codul sursă necesar la fragmentul de mai jos pentru ca printf să afișeze în consolă.

```
int p[2];  
pipe(p);  
dup2(p[1], 1);  
printf("asdf\n");
```

A:

```
#include <stdio.h>
```

```
#include <unistd.h>

int main() {
    int p[2];
    pipe(p);
    if (fork() == 0) {
        // Proces copil
        close(p[1]); // Închide capătul de scriere
        dup2(p[0], 0); // Redirectează capătul de citire la stdin
        char buffer[256];
        fgets(buffer, sizeof(buffer), stdin);
        printf("%s", buffer);
        close(p[0]);
    } else {
        // Proces părinte
        close(p[0]); // Închide capătul de citire
        dup2(p[1], 1); // Redirectează stdout la capătul de scriere
        printf("asdf\n");
        close(p[1]);
    }
    return 0;
}
```

Întrebarea 118

Q:

Care sunt elementele unei adrese virtuale în alocarea paginată-segmentată și ce tabele sunt implicate în calculul adresei fizice?

A:

Elementele unei adrese virtuale sunt (s, p, d), unde:

- s = numărul segmentului

- p = numărul paginii virtuale în segment
- d = deplasamentul (offset) în pagină

Tabelele implicate sunt:

- tabela de segmente a procesului (pentru fiecare segment indică o tabelă de pagini)
- tabela de pagini (pentru fiecare segment)

Deci adresa fizică se calculează: indexul s din tabela de segmente $\rightarrow$ ne dă tabela de pagini a segmentului; indexul p din această tabelă $\rightarrow$ ne dă numărul cadrului fizic; combinat cu $d \rightarrow$ adresa fizică.

Întrebarea 119

Q:

Dați 4 expresii regulate care specifică de câte ori ar trebui să apară expresia anterioară.

a^* $\rightarrow$ de 0 sau mai multe ori

a^+ $\rightarrow$ de 1 sau mai multe ori

$a?$ $\rightarrow$ de 0 sau 1 ori

A:

$a\{x,y\}$ $\rightarrow$ între x și y ori, inclusiv. Dacă x este omis, este 0. Dacă y este omis, este infinit.

Întrebarea 120

Q:

Un proces deschide un FIFO pentru scriere și este singurul proces din sistem care va folosi sau va fi folosit acel FIFO. Când își va termina procesul execuția?

A:

Procesul nu își va termina niciodată execuția, deoarece va fi blocat pentru totdeauna. Procesul va fi blocat până când alt proces deschide FIFO-ul pentru citire. Citirea și scrierea într-un FIFO sunt operații complementare; dacă nu există proces care să citească din FIFO, procesul care scrie va fi blocat.

Întrebarea 121

Q:

4 reguli pentru expresii regulate care arată de câte ori trebuie să se repete expresia anterioară:

A:

a* -> de 0 sau mai multe ori
a+ -> de 1 sau mai multe ori
a? -> de 0 sau 1 ori
a{x,y} -> între x și y ori, inclusiv

Întrebarea 122

Q:

3. Un proces deschide fifo pt citire si e singurul proces ce va folosi vreodata acel fifo, cand isi va incheia executia?

A:

Niciodata, pentru ca acesta asteapta un alt proces pe cealalta parte a fifo-ului sa citeasca ce scrie, dar acesta

nu va exista.

Întrebarea 123

Q:

1. scrieti un grep care ia grupurile de cate 2 cuvinte, separate de un singur spatiu, care sunt formate doar din litere mici si fiecare cuvant contine cel putin 2 vocale

A:

```
grep -Eo '[bcdfghijklmnpqrstvwxyz]*[aeiou][bcdfghijklmnpqrstvwxyz]*[aeiou]
[bcdfghijklmnpqrstvwxyz]* [bcdfghijklmnpqrstvwxyz]*[aeiou][bcdfghijklmnpqrstvwxyz]*[aeiou]
[bcdfghijklmnpqrstvwxyz]*' grep.txt
```

```
"</[a-z]*[aeiou][a-z]*[aeiou][a-z]*/> </[a-z]*[aeiou][a-z]*[aeiou][a-z]*/>"
```

Întrebarea 124

Q:

2. scrieti 2 grep uri care iau liniile care nu au numarul de caractere multiplu al lui 3

A:

```
grep -E -v "^(...)*$" grep.txt
grep -E -v "^(.{3})*$" grep.txt
grep -E -v "^(...)*$"
```

```
grep -E -v -i "^(...)*$"da
```

Întrebarea 125

Q:

3. scrieti un sed care inlocuieste prima aparitie a caracterului A cu caracterul B

A:

sed "s/A/B/" (de pe fiecare linie?)

```
sed -E "s/([A]*) (A)(.*)\1B\3/gi" a.txt
```

```
sed -E "s/A/B/i"
```

Întrebarea 126

Q:

4. scrieti un awk care afiseaza liniile care au primul cuvant identic cu ultimul cuvant si al caror penultim cuvant are numar par de caractere

A:

```
awk 'NF > 1 && $1 == $NF && length($(NF - 1)) % 2 == 0 {print $0}'
```

Întrebarea 127

Q:

5. scrieti 3 moduri de a crea un fisier gol

A:

nano file

> file

touch file

echo "" > a.txt

touch a.txt

echo "" > a.txt

```
echo "" 2> a.txt
```

Întrebarea 128

Q:

6. scrieti 5 moduri de a verifica daca un string este gol(cu test)

A:

```
if test -z "$string";
```

```
if ! test -n "$string"
```

```
if test "$string" == "";
```

```
if test ${#string} -eq 0;
```

```
if test ${#A} -lt 1;
```

1) if test -z \$A

2) if test \$A=""

3) if test \${#A} -eq 0

4) if test \${#A} -lt 1

5) if test ! \${#A} -gt 0

6) if test \$(echo \$A | grep -E -c "^\$") -eq 1

7) if \$(echo \$A | grep -E -q "^\$")

8) if test \$(echo \$A | awk '{print length}') -eq 0

Întrebarea 129

Q:

7. afisati ierarhia proceselor a urmatoarei cod:

```
for(int i = 0; i < 3; i++)  
    if (fork() != 0)  
        wait();
```

A:

Ierarhia este un LANȚ: $P \rightarrow C1 \rightarrow C2 \rightarrow C3$

Explicație:

- i=0: P face fork → C1. P (fork != 0) face wait. C1 (fork == 0) continuă bucla.

- i=1, în C1: fork → C2. C1 face wait. C2 continuă.

- i=2, în C2: fork → C3. C2 face wait. C3 continuă.
- C3 iese din buclă și se termină.
- C2 e deblocat din wait, se termină.
- C1 e deblocat, se termină.
- P e deblocat, se termină.

Fiecare proces are exact un copil, formând un lanț.

Întrebarea 130

Q:

Ce afișează codul:

```
execlp("expr", "expr", "a", "+", "1", NULL);  
printf("xyz\n");
```

A:

Întrebarea originală (cu OCR-uri): `execlp("expr", "expr", "a", "+", "1", NULL); printf("xyz\n");`

Răspuns: programul afișează: `expr: non-integer argument`

Justificare:

- `execlp("expr", "expr", "a", "+", "1", NULL)` execută comanda `'expr a + 1'`
- `expr` nu poate aduna `"a"` (string) cu `1` (număr) – afișează eroare pe `stderr`
- `exec` înlocuiește procesul, deci `printf`-ul NU se execută

Output: mesaj de eroare de la `expr` (`"expr: non-integer argument"` sau similar).

Întrebarea 131

Q:

9. schitați o implementare a funcțiilor `popen` și `pclose`

A:

`popen`:

- creează un pipe
- face fork procesului
- copil: închide capătul de citire al pipe-ului, redirectează `stdout` către pipe și execută comanda cu `execlp`
- părinte: închide capătul de scriere al pipe-ului, returnează un pointer `FILE*` pentru citire

pclose:

- închide pointer-ul de fișier
- așteaptă procesul copil
- returnează statusul de ieșire al copilului

Implementare:

```
FILE* mypopen(char *cmd, char *type) {  
    int p[2];  
    pipe(p);  
    if ((child_pid = fork()) == 0) {  
        if (type[0] == 'r') {  
            close(p[0]);  
            dup2(p[1], STDOUT_FILENO);  
        } else {  
            close(p[1]);  
            dup2(p[0], STDIN_FILENO);  
        }  
        execlp("sh", "sh", "-c", cmd, NULL);  
        exit(1);  
    }  
    if (type[0] == 'r') {  
        close(p[1]);  
        return fdopen(p[0], type);  
    } else {  
        close(p[0]);  
        return fdopen(p[1], type);  
    }  
}
```

```
int mypclose(FILE *fp) {  
    int status;  
    fclose(fp);  
    waitpid(child_pid, &status, 0);  
    return status;  
}
```

Întrebarea 132

Q:

10. cate FIFO pot fi deschise de catre un fisier daca fiecare dintre acele FIFO-uri va avea capatul celalalt deschis de catre un alt proces?

A:

Depinde de limita descriptorilor de fișier ai sistemului, care poate fi verificată folosind "ulimit -n". Este important să deschidem corect și în aceeași ordine FIFO-urile, deoarece altfel poate duce la deadlock-uri. Dacă deschiderea este corectă, limita este cea a descriptorilor de fișier ai sistemului.

Întrebarea 133

Q:

11. cand am dori sa folosim execl si cand am dori sa folosim execv?

A:

Folosim execl: când avem un număr fix și cunoscut de argumente.

Folosim execv: când numărul de argumente este dinamic sau determinat la runtime.

Întrebarea 134

Q:

12. definiti notiunea de sectiune critica

A:

O secțiune critică este o parte a programului unde avem resurse partajate, cum ar fi variabile de memorie, care pot fi accesate și modificate. Este important să gestionăm programul corect pentru a evita race conditions și deadlock-uri, folosind de exemplu excluderea mutuală (mutex).

Întrebarea 135

Q:

14. care sunt consecintele inlocuirii lui `pthread_mutex_lock` cu `sem_post` in cod?

A:

Înlocuirea `pthread_mutex_lock` cu `sem_post` ar duce la haos, deoarece `sem_post` incrementează semaforul, permițând oricărui thread să intre în secțiunea critică, eșuând astfel în prevenirea accesului concurent la resursele partajate. Acest lucru ar putea duce la race conditions și corupere de date.

Întrebarea 136

Q:

15. definiți un semafor binar și explicați cum funcționează

A:

Un semafor binar este un mecanism de sincronizare care poate avea doar două valori: 0 și 1.

Operații:

- wait (P / `sem_wait`): decrementează semaforul (de la 1 la 0). Dacă semaforul e deja 0, thread-ul se blochează.

- signal (V / `sem_post`): incrementează semaforul (de la 0 la 1) și trezește un thread care aștepta.

Funcționează ca un mutex: doar un singur thread poate "intra" la un moment dat.

Întrebarea 137

Q:

17. scrieți un mod de a preveni deadlock

A:

- Folosiți o ordine consistentă de blocare

- Evitați blocaje imbricate

- Folosiți algoritmi de detectare a deadlock-urilor

- Folosiți timeout-uri pentru blocare

Întrebarea 138

Q:

18. prin ce stare (gen ready, wait, swap, etc) trece un proces cand apelam pthread_join?

A:

Stare inițială: thread-ul este în starea de execuție (RUNNING) și apelează pthread_join.

Așteptare terminare: thread-ul care apelează pthread_join trece în starea blocată (BLOCKED/WAIT), așteptând ca thread-ul țintă să se termine.

Thread-ul țintă se termină: când thread-ul țintă își completează execuția, trece în starea TERMINATED.

Deblocarea: thread-ul care aștepta este deblocat și revine în starea READY (apoi RUNNING).

Întrebarea 139

Q:

19. daca avem B drept block size si A drept address size, cate adrese o sa aiba un double indirect dintr-un i-node?

A:

$(B/A)^2$ adrese (sau blocuri de date).

Un bloc are B octeți, o adresă are A octeți, deci un bloc conține B/A adrese.

Indirectarea dublă funcționează astfel:

- Un pointer din i-node spre un bloc cu B/A adrese
- Fiecare dintre acele B/A adrese indică spre alt bloc cu B/A adrese
- Fiecare dintre acele adrese indică spre un bloc de date

Total: $(B/A) \times (B/A) = (B/A)^2$

Întrebarea 140

Q:

20. ce se intampla cu continutul directorului in care montam o partitie?

A:

Conținutul existent al directorului în care montezi partiția devine temporar inaccesibil. După montare, orice acces la director accesează de fapt rădăcina partiției montate.

Întrebarea 141

Q:

Dați o expresie regulată care se potrivește cu orice secvență de lungime pară de cuvinte cu litere mici separate prin spații, dacă pentru fiecare cuvânt lungimea sa și poziția sa în secvență sunt fie ambele impare, fie ambele pare. Ex: al 5-lea cuvânt trebuie să aibă lungime impară, iar al 16-lea trebuie să aibă lungime pară.

A:

Pentru cuvinte cu litere mici separate prin spații, unde cuvântul de pe poziția impară are lungime impară și cel de pe poziția pară are lungime pară:

$$^{([a-z]([a-z][a-z])^* [a-z][a-z]([a-z][a-z])^*)+[a-z]([a-z][a-z])^* [a-z][a-z]([a-z][a-z])^*}$$$

Explicație:

- $[a-z]([a-z][a-z])^*$ = cuvânt de lungime impară (1, 3, 5...)

- $[a-z][a-z]([a-z][a-z])^*$ = cuvânt de lungime pară (2, 4, 6...)

- Le combinăm în perechi (impar, par) repetate, pentru a obține lungime pară a secvenței.

Întrebarea 142

Q:

Câte FIFO-uri poate deschide un proces pentru citire, dacă FIFO-urile sunt și vor fi folosite de alte procese doar pentru scriere?

A:

Numărul maxim este limitat doar de numărul de file descriptors permis sistemului (verificabil cu "ulimit -n").

Condiția importantă: pentru fiecare FIFO deschis pentru CITIRE, trebuie să existe cel puțin un proces care să-l deschidă pentru SCRIERE — altfel apelul open() se blochează.

Dacă procesele de scriere folosesc fiecare FIFO-urile în aceeași ordine ca procesul nostru, putem deschide câte vrem (până la limita ulimit -n).

Întrebarea 143

Q:

Explicați de ce descriptorul de fișier returnat de popen trebuie închis cu pclose în loc de fclose.

A:

popen creează un proces copil prin fork + exec și returnează un FILE* către pipe-ul de comunicare.

Dacă închidem cu fclose:

- Doar stream-ul de date se închide
- Procesul copil rămâne zombie (nu îl așteaptă nimeni)
- Resursele copilului nu se eliberează corect

pclose:

- Închide stream-ul
- Apelează wait() pentru procesul copil (evită zombi)
- Returnează exit status-ul comenzii

Deci pclose face curățenia completă, iar fclose lasă în urmă un zombie.

Întrebarea 144

Q:

Ce va afișa fragmentul de mai jos? Justificați răspunsul.

```
execl("expr", "expr", "1", "+", "1", NULL);  
execlp("echo", "echo", "3", NULL);  
printf("4\n");
```

A:

Output: 3

Justificare:

- execl("expr", ...) – eșuează pentru că "expr" nu este cale absolută și execl NU caută în PATH. Programul continuă.

- `execvp("echo", "echo", "3", NULL)` – reușește (`execvp` caută în `PATH`), înlocuiește procesul curent cu `echo`, care afișează "3" și se termină.
 - `printf("4\n")` – NU se execută, pentru că `execvp` deja a înlocuit procesul.
-

Întrebarea 145

Q:

Dați trei moduri de a afla dimensiunea unui fișier din linia de comandă Linux.

A:

Trei moduri de a afla dimensiunea unui fișier:

1. `ls -l` fișier (a 5-a coloană din output)
2. `wc -c` fișier (numără octeții)
3. `du -h` fișier (utilizare pe disc, lizibil)

Alternative:

- `stat -c%s` fișier
 - `ls -lh` fișier
 - `find . -name fișier -printf "%s\n"`
-

Întrebarea 146

Q:

Scrieți două comenzi SED care afișează liniile unui fișier ștergând prima secvență ne-vidă de litere mici.

A:

```
sed 's/[a-z]\+//' fișier
```

```
sed -E 's/[a-z]\+//' fișier
```

Explicație:

- `[a-z]\+` = una sau mai multe litere mici consecutive
 - `s/.../...` = substituție (implicit doar prima apariție per linie)
 - Cu `-E` (extended regex) nu trebuie escape pe `+`; fără `-E` folosim `\+`
-

Întrebarea 147

Q:

Scrieți o comandă AWK care afișează suma tuturor numerelor dintr-un fișier text ale cărei linii constau din secvențe de cifre separate prin spații.

A:

```
awk '{for(i=1; i<=NF; i++) s += $i} END {print s}' fisier
```

Sau mai simplu, dacă fiecare linie are sumă pe coloana:

```
awk '{for(i=1; i<=NF; i++) s += $i} END {print s}' fisier
```

Explicație:

- NF = numărul de câmpuri pe linia curentă
 - Bucula parcurge toate câmpurile și adună
 - END {} se execută o singură dată la final
-

Întrebarea 148

Q:

Dați trei comenzi GREP care afișează liniile unui fișier care constau exclusiv dintr-o secvență ne-vidă de litere și cifre alternante (ex: a0g sau 1r5m).

A:

```
grep -E '^[a-z][0-9])+$|^[0-9][a-z])+$' fisier
```

```
grep -E '^[a-z0-9]+$' fisier | grep -vE '[a-z]{2}|[0-9]{2}'
```

```
egrep '^[a-z][0-9])+$|^[0-9][a-z])+$' fisier
```

Explicație:

- ^...\$ = linia întreagă
 - ([a-z][0-9])+ = perechi literă-cifră repetate
 - | = sau
 - ([0-9][a-z])+ = perechi cifră-literă repetate
-

Întrebarea 149

Q:

Ce tranziție de stare va suferi un proces când apelează sem_wait și în ce condiții? Justificați răspunsul.

A:

Procesul trece din starea RUNNING în starea WAITING (blocat) atunci când:

- Apelează `sem_wait` pe un semafor cu valoarea 0 (sau care ar deveni negativ după decrementare).

Dacă semaforul are valoarea > 0 , `sem_wait` doar decrementează valoarea și procesul continuă să ruleze (rămâne în RUNNING).

Motiv: dacă procesul ar aștepta activ (busy-wait), ar irosi un core fără să facă nimic. SO îl pune în WAIT și rulează alt proces.

Întrebarea 150

Q:

Ce s-ar putea întâmpla dacă funcția `f` ar fi executată de mai multe thread-uri simultane? De ce?

```
pthread_mutex_t m[2];
void* f(void* p) {
    int id = (int)p;
    pthread_mutex_t* first = &m[id % 2];
    pthread_mutex_t* second = &m[(id + 1) % 2];

    pthread_mutex_lock(first);
    pthread_mutex_lock(second);
    pthread_mutex_unlock(second);
    pthread_mutex_unlock(first);
}
```

A:

Riscul este DEADLOCK.

Thread-urile cu id par ($id \% 2 = 0$) blochează mai întâi `m[0]`, apoi `m[1]`.

Thread-urile cu id impar ($id \% 2 = 1$) blochează mai întâi `m[1]`, apoi `m[0]`.

Dacă două thread-uri (unul par, unul impar) rulează simultan:

- Thread A (id par) prinde `m[0]`
- Thread B (id impar) prinde `m[1]`
- Thread A încearcă să prindă `m[1]` → blocat
- Thread B încearcă să prindă `m[0]` → blocat

→ Niciunul nu poate avansa → DEADLOCK

Cauza fundamentală: nu se respectă o ordine fixă de achiziție a mutex-urilor.

Întrebarea 151

Q:

Care va fi efectul înlocuirii apelurilor la `pthread_mutex_lock` cu apeluri la `sem_post`?

A:

Apariție de race conditions și corupere de date.

`sem_post` incrementează valoarea semaforului, dar NU asigură excluderea mutuală. Mai multe thread-uri pot intra în "secțiunea critică" simultan, ceea ce duce la:

- Modificări concurente ale resurselor partajate
- Rezultate impredictibile

În plus, dacă `pthread_mutex_unlock` rămâne în cod, va apela `unlock` pe un mutex care nu a fost niciodată blocat → comportament nedefinit.

Întrebarea 152

Q:

Dați trei apeluri de funcție care asigură excluderea mutuală.

A:

Trei apeluri care asigură excluderea mutuală:

1. `pthread_mutex_lock(&m)`
 - Cel mai folosit. Blochează un mutex, un singur thread poate intra.
 2. `sem_wait(&s)` cu un semafor binar (inițializat cu 1)
 - Echivalent funcțional cu `mutex_lock`.
 3. `pthread_rwlock_wrlock(&rw)`
 - Blocaj de scriere exclusiv pe un `rwlock` – niciun alt thread (nici cititor, nici scriitor) nu mai poate intra.
-

Întrebarea 153

Q:

Ce conține un fișier de tip director în sistemul de fișiere Linux? - 2, 8

A:

Un fișier de tip director conține o listă de intrări (directory entries), fiecare având:

- Numele fișierului (sau subdirectorului)
- Numărul i-node-ului asociat

Intrările speciale:

- "." (punct) — directorul însuși (i-node-ul lui propriu)
- ".." (două puncte) — directorul părinte
- Intrările obișnuite — numele fișierelor/subdirectoarelor din director și i-node-urile lor.

Observație: numele fișierului NU e stocat în i-node, ci doar în director. I-node-ul conține metadata (drepturi, dimensiune, pointeri spre date).

Întrebarea 154

Q:

Se da fișierul a.log de mai jos în care un server concurent a scris evenimentele aparute pe parcursul rezolvării diferitelor cereri. Fiecare eveniment apare pe o singură linie și are formatul: data ora tip cerere detalii. Scrieți un script Shell UNIX care separă liniile din a.log în fișiere separate (cate unul pentru fiecare cerere) și afișează un top al cererilor în funcție de numărul de erori aparute în procesarea ei.

A:

```
#!/bin/bash

# Separă liniile din a.log în fișiere pe tip de cerere
while IFS= read -r linie; do
    # Format: data ora tip cerere detalii
```

```
tip=$(echo "$linie" | awk '{print $3}')
```

```
echo "$linie" >> "log_$tip.txt"
```

```
done < a.log
```

```
# Variantă mai eficientă cu awk:
```

```
awk '{print >> "log_" $3 ".txt"}' a.log
```

Întrebarea 155

Q:

Dați o expresie regulată care acceptă orice secvență de lungime pară de cuvinte (formate din litere) separate prin spații, cu condiția ca pentru fiecare cuvânt să lungimea și poziția în secvență să fie ambele pare sau impare. Numărarea cuvintelor începe de la 1. Ex: al 5-lea cuvânt trebuie să aibă lungime impară, iar al 16-lea să aibă lungime pară.

A:

La fel ca la întrebarea 123, varianta echivalentă:

```
^([a-z]([a-z][a-z])* [a-z][a-z]([a-z][a-z])* )+[a-z]([a-z][a-z])* [a-z][a-z]([a-z][a-z])* )*$
```

Ideea cheie:

- Cuvânt pe poziție impară → lungime impară → `[a-z]([a-z][a-z])*`
- Cuvânt pe poziție pară → lungime pară → `[a-z][a-z]([a-z][a-z])*`
- Toate perechile (impar, par) compun secvența de lungime pară.

Întrebarea 156

Q:

Dați trei comenzi GREP care afișează dintr-un fișier liniile formate exclusiv dintr-o secvență nevidă de litere și cifre alternativ (ex: a0g sau 1r5m).

A:

```
grep -E '^[a-z][0-9])+$|^[0-9][a-z])+$' fisier
```

```
egrep '^[a-z][0-9])+$|^[0-9][a-z])+$' fisier
```

```
grep '^[a-z][0-9]\\))+$|^[0-9][a-z]\\))+$' fisier
```

Doar litere și cifre alternativ, secvență nevidă, întreaga linie.

Întrebarea 157

Q:

Scrieți două comenzi SED care afișează liniile unui fișier ștergând prima secvență nevidă de litere mici.

A:

```
sed 's/[a-z]\+//' fisier
```

```
sed -E 's/[a-z]\+//' fisier
```

Șterge prima secvență nevidă de litere mici de pe fiecare linie. (Fără /g, doar prima apariție.)

Întrebarea 158

Q:

Scrieți o comandă AWK care afișează suma tuturor numerelor dintr-un fișier text ale cărei linii sunt formate din secvențe de cifre separate prin spații.

A:

```
awk '{for(i=1; i<=NF; i++) s += $i} END {print s}' fisier
```

Parcurge toate câmpurile de pe fiecare linie, adună totul, afișează suma la final.

Întrebarea 159

Q:

Dați trei moduri de a afla dimensiunea unui fișier în linia de comandă Linux.

A:

Trei moduri de a afla dimensiunea unui fișier:

1. `ls -l fisier` (a 5-a coloană este dimensiunea în octeți)
2. `wc -c fisier` (numără octeții)
3. `du -h fisier` (utilizarea pe disc, dimensiune lizibilă)

Alternative: `stat -c%s fisier, ls -lh fisier (lizibil), find . -name fisier -printf "%s\n"`

Întrebarea 160

Q:

Scrieți o condiție Shell UNIX care verifică dacă un fișier există și utilizatorul curent are vreo permisiune asupra lui.

A:

```
if [ -e fisier ] && { [ -r fisier ] || [ -w fisier ] || [ -x fisier ]; };  
then
```

```
    echo "Există și am permisiuni"
```

```
fi
```

Sau mai compact:

```
[ -e fisier ] && [ -r fisier -o -w fisier -o -x fisier ]
```

Explicație:

- -e: există
 - -r: are permisiune de citire
 - -w: are permisiune de scriere
 - -x: are permisiune de executare
-

Întrebarea 161

Q:

Ce va tipări fragmentul de cod de mai jos? Justificați răspunsul.

```
execl("expr", "expr", "1", "
```

```
» NULL);
```

```
NULL) ;
```

```
execlp("echo"  
printf("4\n");
```

```
"echo",
```

A:

Răspuns: 3

execl nu va executa nimic, deoarece are nevoie de calea completă, iar "expr" nu o are, deci comanda nu poate fi executată.

Așa că trecem la execlp, care va afișa "3".

execlp înlocuiește procesul curent, deci nu va afișa "4".

Pentru justificare: execl necesită calea completă către executabil (de exemplu execl("/bin/expr", "expr", "1", "+", "1", NULL)). Aici avem doar numele comenzii, deci execl eșuează. Programul continuă la execlp, afișează "3", iar din moment ce execlp înlocuiește imaginea procesului și se termină la finalizare, niciun cod ulterior nu mai este executat.

Întrebarea 162

Q:

De ce descriptorul de fișier returnat de popen trebuie închis cu pclose și nu cu fclose?

A:

Vezi răspunsul de la întrebarea 125 (popen vs fclose).

Pe scurt: popen creează un proces copil cu fork+exec; pclose închide stream-ul ȘI apelează wait() pentru copil. fclose ar lăsa procesul ca zombie.

Întrebarea 163

Q:

Câte FIFO-uri poate deschide pentru citire un process, dacă FIFO-urile sunt și vor fi întotdeauna folosite de alte procese doar pentru scriere?

A:

Vezi răspunsul de la întrebarea 124. Pe scurt: câte FIFO-uri permite limita sistemului (ulimit -n), respectând ordinea de deschidere pentru a evita deadlock-uri.

Întrebarea 164

Q:

Care va fi efectul înlocuirii apelurilor la pthread_mutex_lock cu apeluri la sem_post?

pthread_mutex_lock with calls to sem_post?

A:

Vezi răspunsul de la întrebarea 132.

Înlocuirea `pthread_mutex_lock` cu `sem_post` duce la haos: `sem_post` incrementează semaforul, nu blochează nimic. Mai multe thread-uri pot intra simultan în secțiunea critică → race conditions și corupere de date.

Întrebarea 165

Q:

Ce se poate întâmpla dacă funcția `f` este rulată de mai multe thread-uri simultane? De ce?

```
pthread_mutex_t m[2];
void* f(void* p) {
    int id = (int)p;
    pthread_mutex_t* first = &m[id % 2];
    pthread_mutex_t* second = &m[(id+1) %
```

```
pthread_mutex_lock(first) ;
pthread_mutex_lock(second);
```

```
pthread_mutex_unlock(second) ;
```

```
pthread_mutex_unlock(first) ;
```

A:

Vezi răspunsul de la întrebarea 131.

Riscul este DEADLOCK pentru că thread-urile cu id par și id impar blochează mutex-urile în ordini opuse. Doi care rulează simultan se pot bloca unul pe altul.

Întrebarea 166

Q:

Prin ce tranziție de stare va trece un process când apelează `sem_wait` și în ce condiții? Justificați răspunsul.

A:

Vezi răspunsul de la întrebarea 130.

Procesul trece din RUN în WAIT dacă apelează `sem_wait` pe un semafor cu valoarea 0. Dacă semaforul are valoare > 0, procesul rămâne în RUN (doar decrementează valoarea).

Întrebarea 167

Q:

2. Scrieți o comandă UNIX Shell care elimină toate caracterele non-literă din fișierul a.txt (de ex: aB59-*! -> aB59).

A:

Comandă UNIX care elimină toate caracterele non-literă din a.txt:

```
sed -E 's/[^a-zA-Z0-9]//g' a.txt
```

Explicație:

- `[^a-zA-Z0-9]` = orice caracter care NU e literă sau cifră
- `s/.../ /g` = înlocuiește global cu nimic (le șterge)

Variantă fără -E:

```
sed 's/[^a-zA-Z0-9]//g' a.txt
```

Pentru a păstra spațiul:

```
sed -E 's/[^a-zA-Z0-9 ]//g' a.txt
```

Întrebarea 168

Q:

3. Scrieți un program AWK care, aplicat pe un fișier care conține cuvinte separate prin spații, calculează numărul mediu de cuvinte pe linie.

A:

SOLUTION

```
BEGIN{
    count=0
    wc=0
}
{
    count++
    wc+=NF
}
END{
    print wc/count
}
```

```
}  
  
avg=words/Lines  
  
int avg
```

Întrebarea 169

Q:

4. Display all the unique file names (without the path) in a given directory and all its hierarchy of subdirectories.

A:

Afișează toate fișierele unice (fără cale) din director și subdirectoarele lui:

```
find . -type f -printf "%f\n" | sort -u
```

Sau echivalent:

```
find . -type f | awk -F/ '{print $NF}' | sort -u
```

Explicație:

- find . -type f → toate fișierele recursiv
- %f în printf-ul lui find → doar numele, fără cale
- sort -u → unice, fără duplicate

Întrebarea 170

Q:

5. Scrieți un script UNIX Shell care calculează numărul mediu de linii din toate fișierele cu extensia .txt din directorul curent.

A:

Script care calculează numărul mediu de linii din toate fișierele .txt:

```
#!/bin/bash  
  
total=0  
  
nr=0  
  
for f in *.txt; do  
    if [ -f "$f" ]; then  
        linii=$(wc -l < "$f")  
        total=$((total + linii))  
        nr=$((nr + 1))  
    fi  
done  
  
avg=$((total / nr))
```

```
fi
done
if [ $nr -gt 0 ]; then
    echo $((total / nr))
else
    echo 0
fi
```

Întrebarea 171

Q:

8. Ce va afișa în consolă fragmentul de cod de mai jos?

```
char* s[3] = {"A", "B", "C"};
for(i=0; i<3; i++) {
    execl("/bin/echo", "/bin/echo", s[i], NULL);
}
```

A:

Va afișa doar "A", deoarece după execuția lui `execl` codul procesului este suprascris cu noul cod generat de `execl`. Familia `exec()` înlocuiește procesul curent cu unul nou. Bucla nu va ajunge niciodată la "B" sau "C".

Întrebarea 172

Q:

9. Ce face apelul de sistem "read" când FIFO-ul conține mai puține date decât se cere să citească?

A:

Va citi datele existente din FIFO.

Întrebarea 173

Q:

10. Ce va afișa codul de mai jos, dacă niciun alt proces nu deschide FIFO-ul "abc"? Justificați răspunsul.

```
int r, w, n = 0;
r = open("abc", O_RDONLY);
n++;
w = open("abc", O_WRONLY);
n++;
printf("%d\n", n);
```

A:

Codul nu va afișa nimic, deoarece va rămâne blocat la primul open, așteptând ca un alt proces să deschidă FIFO-ul la capătul de scriere. După deschiderea FIFO-ului pentru citire, programul va aștepta ca celălalt proces să deschidă FIFO-ul pentru scriere.

Întrebarea 174

Q:

11. Ce se întâmplă cu un proces între momentul în care se termină și momentul în care părintele apelează wait?

A:

Va intra într-o stare de zombie, neexecutând cod dar ocupând un PID. Când rulezi "ps -ef", va apărea ca <defunct>.

Între momentul în care procesul se termină și momentul în care părintele apelează wait, procesul devine zombie.

Întrebarea 175

Q:

17. Câte blocuri de date pot fi referențiate prin indirectarea dublă a unui i-node, dacă un bloc conține N adrese către alte blocuri?

A:

$N \times N = N^2$ blocuri de date.

Explicație: indirectarea dublă funcționează astfel:

- I-node-ul are un pointer spre un bloc de indirectare dublă
 - Acel bloc conține N adrese (pointeri) spre alte blocuri
 - Fiecare dintre acele N blocuri conține N adrese către blocuri de date
 - Total: $N \times N = N^2$ blocuri de date
-

Întrebarea 176

Q:

20. Adăugați instrucțiunile necesare la fragmentul de cod de mai jos, astfel încât stdin-ul comenzii /bin/pwd să fie citit din PIPE-ul p.

```
int p[2];
pipe(p);
if(fork() == 0) {
    execl("/bin/pwd", "/bin/pwd", NULL);
    exit(0);
}
```

A:

```
int p[2];

pipe(p);

if (fork() == 0) {

    close(p[1]); // închidem capătul de scriere în copil

    dup2(p[0], 0); // redirectăm capătul de citire la stdin

    execl("/bin/pwd", "/bin/pwd", NULL);

    exit(0);

}

close(p[0]); // în părinte închidem capătul de citire
// scriem în p[1] datele pentru pwd, apoi close(p[1])
```

Întrebarea 177

Q:

1.

Scrieti o comanda UNIX care afiseaza toate liniile din fisierul a.txt care contin cel putin un num

ar binar

multiplu de 4 cu cinci sau mai multe cifre (ex: 010100).

A:

```
grep -E '[0-1]{3,}[0]{2}' a.txt
```

Explicatie: ultimele 2 cifre din fiecare nr binar trebuie sa fie 0 (ele reprezinta suma cu 2 si 1, daca ele sunt 1 atunci nu e multiplu

de 4). In rest, cifrele din stanga pot fi in numar de 3 sau mai mare decat 3 (ca zice cu cinci s au mai multe cifre).

Întrebarea 178

Q:
2.

Scrieti o comanda UNIX care inverseaza toate perechile de cifra impara urmata de vocala

A:
echo a23e8i97u3 | sed -E 's/([13579])([aeiou])\2\1/gi'

Explicatie: primul caracter sa fie oricare din lista [13579], al doilea oricare din lista vocal
elor, dupa dai swap global netinand
cont de majuscule

Întrebarea 179

Q:
3. Scrieti o comanda UNIX care afiseaza toate scorurile de fotbal unice (ex: 4-0) care apar in
fisierul a.tx
t.

Numarul de goluri poate avea maximum doua cifre

A:
cat a2.txt | sed 's/ /\n/g' | uniq -u | grep -E '^[0-9]{1,2}-[0-9]{1,2}\$'
Explicatia: inlocuiesti spatiul cu enter, ca sa ai fiecare scor pe o linie.
folosesti unig -u ca sa obtii doar liniile unice , apoi grep
pentru a respecta conditiile.

Întrebarea 180

Q:

4

. Afisati numarul de procese ale fiecarui utilizator activ din sistem

A:

Comandă:

```
who | awk '{print $1}' | sort | uniq | awk '{print "ps -u \"$1\" | wc -l"}'  
| /bin/sh
```

Explicație: who pentru a afla utilizatorii activi, sort și uniq ca să-i ai o singură dată. La awk pui delimitatorul să fie spațiu pentru o afișare frumoasă, apoi printezi utilizatorul și apoi dai comanda system ca să afli câte procese are.

Întrebarea 181

Q:

5

. Scrieti un script Shell UNIX care calculeaza media de fisiere cu extensia .txt per director din directorul

curent si toate subdirectoarele lui.

A:

```
#!/bin/bash
```

```
nr_txt=0
```

```
nr_dir=0
```

```
for f in $(find $(pwd) -type f); do
```

```
    if [[ "$f" == *.txt ]]; then
```

```
        echo "$f este fisier .txt"
```

```
        nr_txt=$((nr_txt + 1))
```

```
    fi
```

```
done
```

```
for d in $(find $(pwd) -type d); do
```

```
    echo "$d este director"
```

```
    nr_dir=$((nr_dir + 1))
```

done

echo "Sunt \$nr_txt fisiere text"

echo "Sunt \$nr_dir directoare"

Întrebarea 182

Q:

6

. Cate procese va crea fragmentul de cod de mai jos, excluzand procesul parinte initial ?

```
f(fork()!=fork()){
```

```
fork();
```

A:

6 copii (7 procese inclusiv părintele).

Execuție pas cu pas:

- P face primul fork() → C1. În P returnează PID(C1), în C1 returnează 0.
- P face al doilea fork() → C2.
- C1 face al doilea fork() → C3.
- Acum se evaluează condiția fork() != fork():
 - În P: PID(C1) != PID(C2) → true → P face fork() → C4
 - În C1: 0 != PID(C3) → true → C1 face fork() → C5
 - În C2: PID(C1) != 0 → true → C2 face fork() → C6
 - În C3: 0 != 0 → false → NU face fork

Copii creați: C1, C2, C3, C4, C5, C6 = 6 procese.

Întrebarea 183

Q:

Ce tipărește în consolă fragmentul de cod de mai jos?

```
char* s[3] = {"A", "B", "C"};
for(i=0; i<3; i++) {
    if(fork() != 0) {
        execl("/bin/echo", "/bin/echo", s[i], NULL);
    }
}
```

A:

Afișează "A", "B", "C" – fiecare pe o linie nouă, în ordine NEDETERMINATĂ.

Analiza:

- La fiecare iterație, `fork()` creează un copil. Doar PĂRINTELE (`fork != 0`) intră în `if` și execută `echo`. Copilul (`fork == 0`) NU intră în `if` și continuă bucla.
- `i=0`: P `fork` → C1. P execută `echo "A"` și se înlocuiește (se termină după). Copilul C1 continuă bucla.
- `i=1`, în C1: `fork` → C2. C1 execută `echo "B"` și se înlocuiește. C2 continuă.
- `i=2`, în C2: `fork` → C3. C2 execută `echo "C"`. C3 continuă, iese din buclă și se termină.

P, C1, C2 rulează `echo` în paralel – ordinea în care apar A, B, C pe ecran depinde de planificator (poate fi BAC, CAB, ABC etc.).

Întrebarea 184

Q:

9

.

Ce face apelul sistem "write" cand in PIPE este spatiu, dar nu suficient pentru cat i se cere sa scrie?

A:

Uneori un `write` reușit poate transfera mai puțini octeți decât numărul cerut (`count`). Astfel de scrieri parțiale pot apărea în diferite contexte, unul dintre ele fiind cel de mai sus.

Scrie câți octeți sunt liberi și returnează câți octeți au fost scriși corect.

Întrebarea 185

Q:

10

. Ce tipare este fragmentul de cod de mai jos dacă niciun alt proces nu deschide FIFO-

ul "abc"?

Justificați răspunsul

```
= open("abc", O_WRONLY);
n=write(r, &k, sizeof(int));
printf("3éd\n", n);
```

A:

Codul se blochează la primul `open` al FIFO-ului "abc" (nu există proces complementar).

open("abc", O_RDONLY) blochează până când există un proces care îl deschide pentru scriere.

open("abc", O_WRONLY) blochează până când există un proces care îl deschide pentru citire.

Cum niciun alt proces nu deschide FIFO-ul, primul open al programului se blochează LA NESFÂRȘIT.

Programul NU AFIȘEAZĂ NIMIC și rămâne suspendat.

(Notă: la încercare de write într-un pipe/FIFO fără cititor → SIGPIPE și EPIPE.)

Întrebarea 186

Q:

12

. Considerati ca functia f este executata simultan de 10 thread-uri. Adaugati liniile de cod

necesare ca

sa asigurati ca n va avea valoarea 10 dupa ce thread-urile isi incheie executia ?

```
int n=0;
pthread_mutex_t mtx=PTHREAD_MUTEX_INITIALIZER;

void* f(void* p) {
    pthread_mutex_lock(&mtx);
    nt
    pthread_mutex_unlock(&mtx);
    return NULL;
}
```

A:

Adaugă cod pentru ca f executată de 10 thread-uri să rezulte în n = 10 garantat:

```
pthread_mutex_t mtx;
int n = 0;

void* f(void* p) {
    pthread_mutex_lock(&mtx);
    n++;
    pthread_mutex_unlock(&mtx);
    return NULL;
}
```

În main:

```
pthread_mutex_init(&mtx, NULL);  
// ... creăm thread-urile  
// ... join-uim thread-urile  
pthread_mutex_destroy(&mtx);
```

FĂRĂ mutex: $n++$ NU e atomic (citire-adunare-scriere), race conditions pot face $n < 10$.

Întrebarea 187

Q:

13. Planificati executia job-urilor urmatoare (date ca Nume/Durata/Termen) incat suma intarzieri

lor job-

urilor sa fie minima: A/22/27, B/2/15, C/4/5

A:

Ordinea optimă: C, B, A — întârziere totală = 1

Joburi:

- A: durată 22, deadline 27
- B: durată 2, deadline 15
- C: durată 4, deadline 5

Verificare ordine C, B, A:

- C: pornește la 0, termină la 4, deadline 5 → întârziere 0
- B: pornește la 4, termină la 6, deadline 15 → întârziere 0
- A: pornește la 6, termină la 28, deadline 27 → întârziere 1

Total: $0 + 0 + 1 = 1$

E optim — în orice altă ordine, întârzierea totală e mai mare.

Întrebarea 188

Q:

15. Care este cea mai prioritara categorie de pagini de memorie din care politica de inlocuire N

RU ar

alege o pagina victima

A:

Clasa 0 (0,0): pagini ne-referențiate și ne-modificate.

NRU selectează pagina care aparține primei clase (clasa 0) care nu este goală. Dacă nu există pagini în clasa 0, selectează prima pagină din clasa 1, și așa mai departe.

Clasele:

- Clasa 0: Ne-referențiate, ne-modificate ($r=0, w=0$) — prioritate maximă pentru evacuare
- Clasa 1: Ne-referențiate, modificate ($r=0, w=1$)
- Clasa 2: Referențiate, ne-modificate ($r=1, w=0$)
- Clasa 3: Referențiate, modificate ($r=1, w=1$)

Deci pagina cea mai prioritară pentru a fi aleasă ca victimă este cea cu biții r și w setați la 0.

Întrebarea 189

Q:

17.

Dându-se 2 cache-uri set-asociative, unul cu 2 seturi de 4pagini si unul cu 4setu

ri de 2pagini, care va

da rezultatele mai bune pt secventa de cereri de pagini: 14,23,1,16,1,23,16,14. Justificati raspu

nsul.

A:

4seturi de 2 pagini.

$$14\%4 = 2$$

23

-> 3

1 -> 1

16

-> 0

1 -> 1

23

-> 3

16

-> 0

$14 \% 4 = 2$

pt 4seturi de 2 pagini. Gr: 0

—

16, 1-1, 2-14, 3-

23

pt 2seturi de 4 pagini . Gr: 0->14, 16 .. 1->23, 1

Depinde de cum merge:

Întrebarea 190

Q:

18. Câte blocuri de date pot fi referite prin tripla-indirectare a unui i-node, dacă un bloc are dimensiunea B și o adresă are dimensiunea A?

A:

$(B/A)^3$ blocuri de date.

Explicație: un bloc conține B/A adrese. Indirectarea triplă are 3 niveluri:

- Nivel 1: pointer spre un bloc cu B/A adrese
- Nivel 2: fiecare adresă spre un bloc cu B/A adrese
- Nivel 3: fiecare adresă spre un bloc cu B/A adrese
- Nivel 4: fiecare adresă spre un bloc de date

Total blocuri de date: $(B/A) \times (B/A) \times (B/A) = (B/A)^3$

Întrebarea 191

Q:

19. Ce se întâmplă cu un link hard când fișierul spre care punctează este șters?

A:

Păstrează datele fișierului șters chiar dacă fișierul nu mai există.

Când creezi un hard link către un fișier text și apoi ștergi fișierul text, nu se întâmplă nimic cu hard link-ul - acesta păstrează datele de la fișierul șters.

Dacă ștergi hard link-ul, fișierul original NU este șters (mai există atât timp cât cel puțin un hard link mai există).

(Un fișier = inode care indică spre un bloc de date.)

Pentru un hard link, i-node-urile indică spre date, deci când ștergem un fișier, datele rămân (atât timp cât există încă referințe la inode).

Întrebarea 192

Q:

1. Scrieți o comandă UNIX Shell care afișează liniile dintr-un fișier a.txt care conțin cuvinte ce încep cu literă mare.

A:

```
grep -E "^[A-Z]" a.txt
```

```
grep -E "\<[A-Z].*\>" a.txt
```

Întrebarea 193

Q:

2. Scrieți o comandă UNIX Shell care inversează în fișierul a.txt toate perechile de cifre vecine.

A:

```
sed -E "s/([0-9])([0-9])\1/gi" a.txt
```

Întrebarea 194

Q:

3. Fișierul a.txt conține pe fiecare linie două numere separate prin spațiu. Scrieți o comandă UNIX Shell care afișează pentru fiecare linie suma numerelor.

A:

```
awk '{print s=$1 + $2 }' number
```

Întrebarea 195

Q:

Afișați doar liniile dintr-un fișier a.txt care apar o singură dată (nu sunt duplicate).

A:

```
sort a.txt | uniq -u
```

Explicație:

- sort sortează liniile (necesar pentru uniq, care lucrează pe linii consecutive)
- uniq -u afișează doar liniile UNICE (-u = unique), adică cele care apar exact o singură dată

Fără -u, uniq afișează prima apariție din fiecare grup. Cu -u, afișează doar grupurile cu o singură linie.

Întrebarea 196

Q:

5. Scrieți un script UNIX Shell care afișează numele fiecărui fișier .txt din directorul curent care conține cuvântul "cat".

A:

Script care afișează numele fiecărui fișier .txt care conține cuvântul "cat":

```
#!/bin/bash
```

```
for f in *.txt; do
```

```
if [ -f "$f" ] && grep -q "\<cat\>" "$f"; then  
    echo "$f"  
fi
```

done

Sau cu find + grep:

```
find . -maxdepth 1 -name "*.txt" -exec grep -l "\<cat\>" {} \;
```

Sau cu grep direct:

```
grep -l "\<cat\>" *.txt
```

Întrebarea 197

Q:

(fără conținut)

A:

Cazul 1: (execl reușește)

→ Output-ul va fi "A", deoarece comanda execl șterge codul inițial și îl înlocuiește cu codul noului program.

Cazul 2: (execl nu reușește)

→ Output-ul va fi "A" și apoi "B", deoarece apelul execl a eșuat și procesul apelant continuă să execute.

Întrebarea 198

Q:

14. Dați un avantaj și un dezavantaj al alocării segmentate față de alocarea paginată.

A:

Dezavantaj: Alocarea segmentată este mai lentă decât cea paginată în privința accesului la memorie.

Avantaj: Alocarea segmentată oferă o protecție mai bună a memoriei, deoarece fiecare segment poate primi drepturi de acces diferite.

Întrebarea 199

Q:

20. Ce este un semafor binar și care este efectul metodei P când este apelată de mai multe procese/thread-uri concurente?

A:

Un semafor binar este restricționat la valorile 0 și 1, putând fi folosit pentru a controla accesul la o singură resursă, în special pentru a asigura excludere mutuală pentru o secțiune critică (este cunoscut și ca mutex lock).

Metoda P (= wait, sleep, down): dacă semaforul este deblocat (valoare = 1), devine blocat (valoare = 0) și apelul se întoarce, permițând thread-ului să continue. Dacă semaforul este blocat (valoare = 0), thread-ul este blocat și plasat într-o coadă Q de thread-uri în așteptare.

Semaforul este inițializat cu 1. Folosește două operații atomice: 1) wait (P), 2) signal (V) pentru sincronizarea proceselor.

Întrebarea 200

Q:

Avantaj si dezavantaj al alocarii cu partiti variabile si alocarea paginata

A:

Avantaj alocare cu partiții variabile (față de paginată):

- Adresa nu trebuie translatată dintr-o adresă virtuală într-una fizică (sau translatarea este foarte simplă), deci accesul la memorie este mai rapid din punct de vedere computațional.
- Nu există fragmentare internă (procesul primește exact cât are nevoie).

Dezavantaj:

- Apare fragmentarea EXTERNĂ: după multe alocări și eliberări, memoria rămâne plină de spații libere mici, neutilizabile.
- Compactarea memoriei (pentru a uni spațiile libere) este costisitoare și nu se poate face online ușor.
- Necesită ca un proces să fie încărcat în memorie continuă (de la o adresă până la alta neîntreruptă), ceea ce e o restricție grea.

Întrebarea 201

Q:

Cand trece un process din starea de RUN in starea de READY

De ce operatiile 1/0 cauzeaza un Proces: 'treaca din starea de RUN in

Mee tee cee etiors couse a process to mons ee 'the

Cum se ore calculul de adresa in alocarea cu partis:
ee nl

A:

Un proces își va schimba starea din RUN în READY după ce a făcut tot ce avea de făcut în timpul alocat pe procesor (după ce expiră cuanta sa de timp pe procesor).

De asemenea, dacă procesul face o operație I/O, va trece din RUN în WAIT (nu READY), deoarece operațiile I/O pot dura mult, iar scheduler-ul rulează alt proces care e în Ready (scheduler-ul este preemptiv, anticipativ).

Întrebarea 202

Q:

(fără conținut)

A:

(fără conținut)

Întrebarea 203

Q:

Implementati programele A, B si C care comunica prin FIFO. The 5 ori, A si B (jucatori) ii trimit lui C (loteria) cate 6 intregi de la 1 la 49 pe care C ii compara cu 6 intregi intre 1 si 49 alesi la pornire. Loteria raspunde jucatorilo x cu numarul de intregi ghiciti. Jucatorii afiseaza de fiecare data cati intregi

au ghicit.

A:

// Loteria (proces C):

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
#include <sys/stat.h>
```

```
#include <time.h>
```

```
#define N 5
```

```

int main() {
    mkfifo("/tmp/a2c", 0666);
    mkfifo("/tmp/b2c", 0666);
    mkfifo("/tmp/c2a", 0666);
    mkfifo("/tmp/c2b", 0666);
    srand(time(NULL));
    int extrase[6];
    for(int i = 0; i < 6; i++) extrase[i] = rand() % 49 + 1;
    int fa = open("/tmp/a2c", O_RDONLY);
    int fb = open("/tmp/b2c", O_RDONLY);
    int wa = open("/tmp/c2a", O_WRONLY);
    int wb = open("/tmp/c2b", O_WRONLY);
    for(int r = 0; r < N; r++) {
        int ja[6], jb[6];
        read(fa, ja, sizeof(ja));
        read(fb, jb, sizeof(jb));
        int ga = 0, gb = 0;
        for(int i = 0; i < 6; i++)
            for(int j = 0; j < 6; j++) {
                if(ja[i] == extrase[j]) ga++;
                if(jb[i] == extrase[j]) gb++;
            }
        write(wa, &ga, sizeof(int));
        write(wb, &gb, sizeof(int));
    }
    close(fa); close(fb); close(wa); close(wb);
    unlink("/tmp/a2c"); unlink("/tmp/b2c");
    unlink("/tmp/c2a"); unlink("/tmp/c2b");
}

```

```
    return 0;
}
// Jucător (A sau B) – schimbă numele FIFO-urilor:
int main() {
    int w = open("/tmp/a2c", O_WRONLY);
    int r = open("/tmp/c2a", O_RDONLY);
    srand(time(NULL));
    for(int i = 0; i < 5; i++) {
        int nums[6];
        for(int j = 0; j < 6; j++) nums[j] = rand() % 49 + 1;
        write(w, nums, sizeof(nums));
        int ghicit;
        read(r, &ghicit, sizeof(int));
        printf("A ghicit: %d\n", ghicit);
    }
    close(w); close(r);
    return 0;
}
```

Întrebarea 204

Q:

Scrieti o expresie regulara care accepta orice lant care contin cel putin trei vocale.

A:

Grep -E '([aeiou].*){3,}' fisier

Întrebarea 205

Q:

Ce va afisa in consola fragmentul de cod de mai jos? Justificati raspunsul.

```
char* s[3] = {"A", "B", "C"};
for(i=0; i<3; i++) {

execl("/bin/echo", "/bin/echo", s[i], NULL);
}
```

Raspundeti mai jos/Write your answer below:

A:

Va afișa doar "A".

Explicație: la prima iterație (i=0), execl("/bin/echo", "/bin/echo", s[0], NULL) reușește. Familia exec() ÎNLOCUIEȘTE procesul curent cu un alt program. Astfel, procesul devine /bin/echo, care afișează "A" și se termină.

Codul de DUPĂ execl (restul iterațiilor buclei for) NU se mai execută niciodată – programul original a fost suprascris.

Pentru a afișa A, B, C ar fi trebuit fork pentru fiecare iterație:

```
if (fork() == 0) {
    execl("/bin/echo", "/bin/echo", s[i], NULL);
    exit(1);
}
wait(NULL);
```

Întrebarea 206

Q:

Care este principiul vecinatatit in privinta incarcarii paginilor unu proces?

A:

Cand incarcam o pagina refetita, incarcam si cateva pagini de langa ea

Întrebarea 207

Q:

Plantficatt Job-urtle de mat jos (date ca Nune/Durata/Termen) incat suna tntarztertlor task-urtlor sa fle minima.

A:

BADC

24+54+3+4

O intarziere de 2 la D si una la C. intarziere totala de 3

Întrebarea 208

Q:

Ce risc ridtca functta f daca este rulata tn mat multe thread-urt stnultane?
Justificatt raspunsul.

```
/* R0: Consideratt ca mutex-it sunt initialtzatt corect */  
Consider that the mutexes are properly tnittaltzed */  
1 mutex_t m[2];  
votd* f(votd* p) {  
  
    tnt td = (int)p;  
  
    pthread_mutex_t* first = am[td x 2];  
  
    pthread_mutex_t* second = am[(tde1) % 2];  
  
    pthread_mutex_lock(first);  
    pthread_mutex_lock(second);  
  
    pthread_mutex_unlock(second);  
    pthread_mutex_unlock(first);  
  
}
```

Raspundett mat jos/Write your answer below:

A:

Deadlock

Lock(0) lock(1)

Lock(1) lock(0)

Si apuca fiecare thread sa faca primul lock.

Întrebarea 209

Q:

Dati un avantaj si un dezavantaj al politicii de plasare First-Fit fata de Worst-Fit.

A:

First Fit vs Worst Fit:

First Fit (primul potrivit):

- Parcurge lista de blocuri libere și folosește PRIMUL bloc suficient de mare
- Avantaj: rapid (nu parcurge toată lista)

- Dezavantaj: lasă multe fragmente mici (fragmentare externă) la începutul memoriei

Worst Fit (cel mai mare):

- Folosește CEL MAI MARE bloc liber disponibil
- Avantaj: după alocare rămâne un fragment mai mare, util pentru alocări viitoare
- Dezavantaj: parcurge mereu toată lista (mai lent); poate consuma blocurile mari prematur

În practică, First Fit este preferat pentru viteza sa.

Întrebarea 210

Q:

Care este cea mai prioritara pagina de memorie pe care politica de inlocuire NRU ar alege-o ca pagina victima?

A:

Politica NRU (Not Recently Used) alege ca pagină victimă din PRIMA CLASĂ NEVIDĂ.

Clasele NRU sunt definite prin biții R (referit) și M (modificat):

- Clasa 0: R=0, M=0 (nereferit, nemodificat) — PRIORITATE MAXIMĂ
- Clasa 1: R=0, M=1 (nereferit, modificat)
- Clasa 2: R=1, M=0 (referit, nemodificat)
- Clasa 3: R=1, M=1 (referit, modificat) — ultima opțiune

NRU alege o pagină aleatoare din PRIMA clasă care nu e goală.

Întrebarea 211

Q:

(fără conținut)

A:

$(B/A)^3$

Întrebarea 212

Q:

(fără conținut)

A:

```
Grep -E -I '[a]{1,}' fisier | grep -E -I -v '[b]{1,}' fisier
```

```
Grep -E -I '^[^b]*a[^b]*$' fisier
```

Întrebarea 213

Q:

Care este numărul maxim de procese fiu create de fragmentul de cod de mai jos care pot coexista simultan?

```
for(i=0; i<7; i++) {  
    if(fork() == 0) {  
        sleep(rand() % 10);  
        exit(0);  
    }  
    if(i % 3 == 0) {  
        wait(0);  
    }  
}
```

A:

5 copii pot coexista simultan.

Analiza pas cu pas pentru părinte:

- i=0: fork → C1 (1 copil). i%3==0 → wait → așteaptă C1 (0 copii).
- i=1: fork → C2 (1 copil). i%3≠0 → nu așteaptă.
- i=2: fork → C3 (2 copii). i%3≠0 → nu așteaptă.
- i=3: fork → C4 (3 copii). i%3==0 → wait → așteaptă un copil să se termine (2 copii).
- i=4: fork → C5 (3 copii). nu așteaptă.
- i=5: fork → C6 (4 copii). nu așteaptă.
- i=6: fork → C7 (5 copii). i%3≠0 → nu așteaptă.

Maxim simultan: 5 copii.

(Numărul exact depinde de ordinea de execuție, dar 5 este maximul teoretic atins după ultimul fork.)

Întrebarea 214

Q:

(fără conținut)

A:

3. Daca de exemplu am 4 core uri as porni de la 4 threaduri si asa stiu ca lucreaza in parallel pe core uri diferite. Masor timpul de executie. Si ma duc cu multiplii de 4 pana cand timpul de rulare creste si se allege nr de threaduri pentru care am obtinut cel mai mic timp de executie

Întrebarea 215

Q:

(fără conținut)

A:

4. Pentru ca dureaza mult si ca sa nu tina ocupat procesorul le punem in wait pana se incarca

Întrebarea 216

Q:

(fără conținut)

A:

5. Adresa executabilului e aceeași cu adresa fizică.

Întrebarea 217

Q:

dar nu contin litera

A:

Întrebare incompletă în deck (text trunchiat OCR).

Pe baza fragmentului: "dar nu contin litera..." probabil despre regex care acceptă linii cu anumite caractere DAR nu conțin o literă specifică.

Pattern generic: linii care conțin 'a' dar nu conțin 'b':

grep -E '^[^b]*a[^b]*\$' fisier

Sau:

grep "a" fisier | grep -v "b"

Întrebarea 218

Q:

'xm/exam/os/account/odir2783/ex2419> cat wememlp.txt

ti un avantaj si un dezavantaj al incarcarii tuturor paginilor unui program la pornirea procesului în compar ie cu incarcarea lor doar cand sunt necesare.

77

A:

Vezi răspunsul de la întrebarea 94.

Încărcarea TUTUROR paginilor la pornire:

- + Avantaj: nu apar page faults în timpul execuției → execuție fluidă
- Dezavantaj: irosim memorie (încărcăm și pagini ce nu vor fi folosite); pornirea e lentă

Încărcarea LA CERERE (demand paging):

- + Avantaj: folosim eficient memoria, pornire rapidă
- Dezavantaj: page faults în timpul execuției → pauze

Întrebarea 219

Q:

Ce va afișa fragmentul de cod de mai jos, considerând că thread-urile se creează fără probleme? Justificați răspunsul.

```
/* Considerați că header-ele necesare sunt incluse aici */
void* f(void* a) {
    printf("%d\n", *(int*)a);
    return NULL;
}

int main() {
    /* Considerați că variabilele necesare sunt declarate aici */
    for(i=0; i<10; i++) {
        pthread_create(&t[i], NULL, f, &i);
    }
    /* Considerați că aici se fac join-urile necesare */
    return 0;
}
```

Răspundeți mai jos:

A:

Output: nu este cert ce vor afișa thread-urile.

Vor fi afișate 10 numere, dar valorile și ordinea sunt impredictibile, pentru că:

- Thread-urile primesc adresa lui i (pointer), nu valoarea

- Variabila `i` este incrementată de bucla principală în paralel cu thread-urile

- Când un thread accesează `*(int*)a`, citește valoarea curentă a lui `i`, care poate fi orice între 0 și 10

Posibile output-uri: "0 1 2 3 4 5 6 7 8 9" (ideal), dar mai probabil ceva ca "3 3 5 7 7 9 10 10 10 10".

Soluție corectă: trimite COPIE a lui `i`, nu pointer:

```
int *p = malloc(sizeof(int)); *p = i; pthread_create(&t, NULL, f, p);
```

Întrebarea 220

Q:

Care sunt elementele unei adrese virtuale în alocarea paginat-segmentată și ce tabele sunt implicate în calcularea adresei fizice?

A:

Adresa virtuală în alocarea paginat-segmentată: (`s`, `p`, `d`)

- `s` = numărul segmentului

- `p` = numărul paginii virtuale în segment

- `d` = deplasamentul (offset) în pagină

Tabelele implicate:

1. Tabela de segmente a procesului — pentru fiecare segment indică o tabelă de pagini

2. Tabela de pagini (una pentru fiecare segment) — pentru fiecare pagină indică numărul cadrului fizic

Calcul: `s` → tabela de segmente → tabela de pagini a segmentului → `p` → numărul cadrului fizic → + `d` → adresa fizică.

Întrebarea 221

Q:

Adăugați codul sursă necesar la fragmentul de cod de mai jos pentru ca `printf` să afișeze în consolă.

```
int p[2];
pipe(p);
dup2(p[1], 1);
printf("asdf\n");
```

Răspundeți mai jos:

A:

```
#include <stdio.h>

#include <unistd.h>

int main() {
    int p[2];

    pipe(p);

    if (fork() == 0) {
        // Proces copil: citește din pipe și afișează
        close(p[1]);

        dup2(p[0], 0); // redirectează stdin la pipe

        char buf[256];

        fgets(buf, sizeof(buf), stdin);

        printf("Copilul a citit: %s", buf);

        close(p[0]);
    } else {
        // Părintele: scrie în pipe
        close(p[0]);

        dup2(p[1], 1); // redirectează stdout la pipe

        printf("asdf\n");

        fflush(stdout);

        close(p[1]);
    }

    return 0;
}
```

Întrebarea 222

Q:

Ce va afișa în consola fragmentul de cod de mai jos? Justificați răspunsul.

```
char* s[3] = ("A", "B", "C");
for(i=0; i<3; i++) {

    execl("/bin/echo", "/bin/echo", s[i], NULL);
    t
```

Răspundeți mai jos/Write your answer below:

A:

Va afișa doar "A".

De ce? La prima iterație, `execl("/bin/echo", "/bin/echo", "A", NULL)` reușește și înlocuiește procesul curent cu `/bin/echo`, care afișează "A" și se termină.

Familia `exec()` înlocuiește procesul curent cu un alt program. Codul de DUPĂ `exec` (restul iterațiilor buclei `for`) nu se mai execută niciodată – programul original a fost suprascris.

Pentru a afișa A, B, C, ar trebui `fork` pentru fiecare iterație:

```
if (fork() == 0) {

    execl("/bin/echo", "/bin/echo", s[i], NULL);

    exit(1);

}

wait(NULL);
```

Întrebarea 223

Q:

Scrieți o expresie regulată care accepte orice linie care conține cel puțin trei vocale.

A:

```
^[^aeiou]*[aeiou][^aeiou]*[aeiou][^aeiou]*[aeiou].*$
```

Sau folosind lookahead-uri (PCRE):

```
^(?=.*[aeiou]){3}).*$
```

Variantă mai simplă cu `grep`:

```
grep -E '([aeiou].*){3}' fisier
```

Explicație:

- [aeiou] = o vocală
 - .* între ele = orice caractere (inclusiv nimic)
 - de 3 ori = cel puțin 3 vocale
-

Întrebarea 224

Q:

Care este principiul vecinătății în privința încărcării paginilor unui proces?

A:

Principiul vecinătății (sau principiul localității):

Un proces va avea probabil nevoie în curând de paginile vecine paginii tocmai accesate. Această observație are două forme:

- Localitate spațială: dacă o adresă este accesată, adresele vecine au șanse mari să fie accesate în curând

- Localitate temporală: o adresă accesată recent are șanse să fie accesată din nou

Aplicare: când încărcăm o pagină în memorie, "preîncărcăm" și vecinii ei (prefetching). Asta reduce numărul de page faults.

Întrebarea 225

Q:

4. Dat fiind un director și ierarhia sa, scrieți o comandă UNIX Shell care afișează toate albumele din el. Un album este un director care conține fișiere cu extensia ".mp3".

A:

Comandă care afișează toate albumele dintr-un director și subdirectoarele lui.

Un album = director care conține fișiere .mp3.

```
find . -type d -exec sh -c 'ls -l "$1"/*.mp3 >/dev/null 2>&1 && echo "$1" _  
{ } \;
```

Sau mai simplu cu find -name:

```
find . -name "*.mp3" -printf "%h\n" | sort -u
```

(%h = directorul ce conține fișierul, sort -u → unic.)

Întrebarea 226

Q:

3. Display a list of all unique appearances of number Pi with 2 or more decimals in a.txt.

A:

```
grep -E-i-o "(3\.14[0-9]*)" a.txt | sort | uniq
```

Întrebarea 227

Q:

2. Display lines of file a.txt by swapping any pairs of vowel followed by odd digit (a23b7a98i3 -> a23b79a83i).

A:

```
sed -E "s/([aeiou])([13579])\2\1/gi" a.txt
```

Întrebarea 228

Q:

1. Display lines in a.txt that contain at least 1 timestamp (hh:mm:ss:sss).

A:

```
grep -E -i "^.*(([0-9]{2}:){3}[0-9]{3}).*" a.txt
```

Întrebarea 229

Q:

5. Scrieți un script UNIX Shell care calculează numărul mediu de linii de cod în toate fișierele cu extensia ".sh" din directorul curent, ignorând liniile de comentariu și liniile care conțin doar spații și taburi.

A:

```
#!/bin/bash
```

```
total=0
```

```
nr=0
```

```
for f in *.sh; do
```

```

if [ -f "$f" ]; then
# Numărăm liniile care NU sunt comentarii sau goale
linii=$(grep -vE '^\s*(#|$)' "$f" | wc -l)
total=$((total + linii))
nr=$((nr + 1))
fi
done
if [ $nr -gt 0 ]; then
echo "Media: $(echo "scale=2; $total / $nr" | bc)"
else
echo "Nu există fișiere .sh"
fi

```

Explicație: grep -vE '^\s*(#|\$)' filtrează liniile care încep cu # (comentarii) sau sunt goale/cu spații.

Întrebarea 230

Q:

Câte procese sunt create?

```

if(fork() || fork()) {
    fork();
}

```

A:

3 procese sunt create (excluzând părintele).

Analiza:

- fork() #1 → creează C1. În părinte returnează PID-ul lui C1 (≠ 0, deci truthy). În C1 returnează 0 (falsy).

- În părinte: condiția primului fork() e truthy, deci || NU evaluează al doilea fork; intră în if și face un nou fork() → creează C2.

- În C1: primul fork() returnează 0 (falsy), deci || EVALUEAZĂ al doilea fork() → creează C3. Apoi în C1 condiția devine PID(C3) (truthy), intră în if și creează încă un fork → C4. În C3, ambele sunt 0, deci condiția e falsă, nu face if.

Total: C1, C2, C3, C4 = 4 procese.

Observație: răspunsul exact depinde de interpretarea exactă a operatorului `||` cu `fork()` – recomandat să verifici cu un program de test.

Întrebarea 231

Q:

7. Câte procese sunt create când procesul părinte apelează `f(3)`?

```
void f(int n) {
    if (n > 0 || fork() == 0) {
        f(n-1);
        exit(0);
    }
    wait(0);
}
```

A:

```
void f(int n) {

    if (n > 0 || fork() == 0) {
        f(n - 1);
        wait(NULL);
    }
}
```

Apel `f(3)`:

- `n=3`: `n>0` e true → intră în if → `f(2)` → wait
- `n=2`: `n>0` → `f(1)` → wait
- `n=1`: `n>0` → `f(0)` → wait
- `n=0`: `n>0` false → `fork() == 0`?
 - Părintele: `fork()` returnează PID copil `> 0` → false → NU intră în if
 - Copilul: `fork()` returnează 0 → true → `f(-1)` → wait...

Pentru `f(-1)`: `n>0` false → `fork()==0`? → tot creează copii recursiv → COPII INFINIȚI!

Programul NU se termină – fork bombs cu copii infinit.

Întrebarea 232

Q:

Ce va afișa codul de mai jos?

```
char* s[3] = {"A", "B", "C"};
for(i=0; i<3; i++) {
    execl("/bin/echo", "/bin/echo", s[i], NULL);
}
```

A:

```
char* s[3] = {"A", "B", "C"};
for(i=0; i<3; i++) {
```

```
execl("/bin/echo", "/bin/echo", s[i], NULL);  
}
```

La prima iterație (i=0): execl reușește → procesul curent e ÎNLOCUIT cu echo A.

Procesul afișează "A" și SE TERMINĂ.

Niciodată NU se ajunge la i=1 sau i=2.

Output: doar "A" (urmat de newline).

(Familia exec înlocuiește imaginea procesului – codul DUPĂ exec nu se mai execută.)

Întrebarea 233

Q:

9. Ce face apelul de sistem "read" când pipe-ul conține mai puține date decât se cere să citească, dar nu este gol?

A:

read() va citi cât poate — adică toate datele disponibile în pipe, chiar dacă sunt mai puține decât a cerut. Returnează numărul efectiv de octeți citiți.

Exemplu: dacă cer read(fd, buf, 100) dar în pipe sunt doar 30 octeți, read va întoarce 30 (cei disponibili) imediat.

DIFERENȚA față de cazul "pipe gol":

- Pipe gol și writer activ: read() BLOCHEAZĂ
- Pipe gol și writer închis: read() returnează 0 (EOF)
- Pipe cu date (chiar dacă mai puține decât cerute): read() returnează imediat ce poate

Întrebarea 234

Q:

10. Ce va afișa codul?

```
int p[2];  
char buf[10];  
int n;  
pipe(p);  
n = read(p[0], buf, 10);  
printf("%d\n", n);
```

A:

iccametiammmntieentinetti

Se va bloca deoarece pipe-urile de scriere nu au fost inchise si read va astepta date de la un scriitor inexistent.

Daca se presupune ca pipe-ul de scriere este inchis, va printa 0.

Întrebarea 235

Q:

11. De ce este un proces zombie o problemă?

A:

Ocupă PID-uri și sistemul se va bloca pentru că nu vor mai fi PID-uri disponibile.

Procesele zombie sunt problematice deoarece, dacă nu sunt "așteptate" cu wait, vor crește în număr, ocupând PID-uri și aducând sistemul într-un punct în care nu mai pot fi create alte procese din cauza lipsei de PID-uri.

Întrebarea 236

Q:

12. f is executed simultaneously by 10 threads. Add the necessary code to ensure n is 10 after threads have completed.

A:

Cod corect pentru a asigura $n = 10$ după 10 thread-uri:

```
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
int n = 0;
```

```
void* f(void* p) {
    pthread_mutex_lock(&mtx);
    n++;
    pthread_mutex_unlock(&mtx);
    return NULL;
}
```

(În main: creezi 10 thread-uri și le join-uești. Mutex-ul garantează că $n++$ e atomic.)

Întrebarea 237

Q:

13. Planificați activitățile astfel încât suma întârzierilor să fie minimă: A/7/13, B/5/9, C/2/4.
(Format: Job/Durată/Termen)

A:

$t = 0$: timpul inițial

C: pornește la 0, termină la 2 (durata 2), termen 4 → întârziere 0

B: pornește la 2, termină la 7 (durata 5), termen 9 → întârziere 0

A: pornește la 7, termină la 14 (durata 7), termen 13 → întârziere 1

Ordinea optimă: CBA, întârziere totală = 1.

Întrebarea 238

Q:

14. Scrieți un avantaj și un dezavantaj al cache-urilor set-asociative față de cache-urile cu acces direct.

A:

Avantaj: Cache-urile set-asociative sunt mai rapide deoarece politica de înlocuire este aplicată rar (șanse mult mai mici de cache thrashing).

Dezavantaj: Sunt de obicei mai lente și mai scumpe de construit din cauza multiplexorului de ieșire și a comparatoarelor adiționale.

În altă formulare:

Avantaj: Cache-urile set-asociative au de obicei rate mai mici de miss decât cache-urile cu acces direct de aceeași capacitate, deoarece au mai puține conflicte.

Dezavantaj: Cache-urile set-asociative sunt mai lente și mai scumpe de construit; cache-urile cu acces direct pot cauza coliziuni de cache (cache thrashing).

Întrebarea 239

Q:

15. Ce pagină are cea mai mare prioritate în politica de înlocuire LRU, când se alege o pagină victimă?

A:

Pagina care a avut cel mai mic număr de solicitări. Dacă folosim contorul de acces: pagina victimă este cea cu cel mai mic contor. Dacă folosim matricea de referințe: pagina victimă este numărul liniei cu cele mai puține 1-uri.

LRU are o matrice $N \times N$ de biți (N = numărul de pagini) care este umplută cu 0 și 1, iar prioritatea cea mai mare la alegerea paginii victimă o au paginile care au suma minimă pe linie.

Întrebarea 240

Q:

16. Date 2 cache-uri set-asociative, unul cu 2 seturi de 4 pagini și unul cu 4 seturi de 2 pagini, care ar performa mai bine pentru următoarea secvență de cereri de pagini: 20, 9, 18, 27, 20, 9, 18, 27.

A:

Pentru cache-ul set-asociativ cu 2 seturi de 4 pagini există multe coliziuni deoarece restul împărțirii poate fi doar 0 sau 1, deci toate paginile vor cădea pe doar 2 seturi.

Cel cu 4 seturi de 2 pagini va performa mai bine, deoarece coliziunile nu se întâmplă atât de des. (Folosind formula $K \bmod N$, unde K = numărul paginii și N = numărul de seturi, cu mai puține seturi se cauzează cache thrashing când iterăm prin coloane.)

Întrebarea 241

Q:

17. Câte blocuri de date pot fi referențiate prin tripla-indirectare a unui i-node, dacă un bloc conține N adrese către alte blocuri?

A:

Indirectarea triplă a unui i-node referă $N \times N \times N = N^3$ blocuri de date.

Sau folosind notația alternativă (B = dimensiune bloc, A = dimensiune adresă):

Numărul de adrese per bloc = B/A

Indirectare triplă = $(B/A)^3$

Explicație:

- Nivelul 1 (i-node → bloc): B/A adrese
 - Nivelul 2: fiecare adresă → bloc cu B/A adrese
 - Nivelul 3: fiecare adresă → bloc cu B/A adrese de date
 - Total blocuri de date: $(B/A) \times (B/A) \times (B/A) = (B/A)^3$
-

Întrebarea 242

Q:

65. Considerând problema producător-consumator cu un buffer de capacitate N. Câte semafoare ați folosi pentru a asigura corectitudinea operațiilor și care ar fi valorile lor inițiale?

A:

Trei semafoare (sau 2 + un mutex):

- un semafor cu valoare inițială 0: indică numărul de poziții ocupate

- un semafor cu valoare inițială N: indică numărul de poziții libere

- un semafor binar (sau mutex) cu valoare 1: asigură accesul exclusiv la buffer când este modificat

Întrebarea 243

Q:

19. Metodă pentru prevenirea deadlock-ului când nu poți evita modificarea concurentă a resurselor.

A:

Metoda pentru prevenirea deadlock-ului:

Stabilește o ordine globală a resurselor (orice ordine, dar fixă) și blochează-le mereu în aceeași ordine în toate thread-urile. Astfel se elimină condiția de "așteptare circulară" — una din cele 4 condiții necesare pentru deadlock.

Exemplu: dacă ai mutex-urile A, B, C, blochează-le mereu în ordinea $A \rightarrow B \rightarrow C$.
Niciodată invers.

Întrebarea 244

Q:

21. Scrieți o comandă UNIX care afișează toate liniile din fișierul a.txt care conțin cel puțin un număr binar care este multiplu de 4 cu 5 sau mai multe cifre (ex: 010100).

A:

Comandă care afișează liniile cu cel puțin un număr binar multiplu de 4:

```
grep -E '\b[01]+00\b' a.txt
```

Sau:

```
grep -E '[01]{3,}[0]{2}' a.txt
```

Explicație: un număr binar este multiplu de 4 dacă ultimele DOUĂ cifre sunt 00.

- \b = limita de cuvânt
 - [01]+ = una sau mai multe cifre binare
 - 00 = ultimele 2 cifre 0 (multiplu de 4)
-

Întrebarea 245

Q:

22. Scrieți o comandă UNIX care inversează toate perechile care conțin o cifră impară urmată de o vocală (a23e8i97u3 -> a2e38i9u73).

A:

aZesolsu/3)).

```
sed -E"s/([13579])([aeiou])/\2\1/gi" a.txt
```

Întrebarea 246

Q:

23. Scrieți o comandă UNIX care afișează toate scorurile unice de fotbal (ex: 4-0) care apar în a.txt. Numărul de goluri poate avea cel mult 2 cifre.

A:

Afișează scoruri unice de fotbal (format x-y, max 2 cifre fiecare):

```
grep -oE '\b[0-9]{1,2}-[0-9]{1,2}\b' a.txt | sort -u
```

Explicație:

- -o = afișează doar partea care match-uește (scorul, nu toată linia)
 - -E = regex extins
 - [0-9]{1,2} = una sau două cifre
 - \b = limita de cuvânt (evită "123-456")
 - sort -u = unic, fără duplicate
-

Întrebarea 247

Q:

24. Print the number of processes of every active user in the system.

A:

Numărul de procese ale fiecărui utilizator activ:

```
ps -ef | tail -n +2 | awk '{print $1}' | sort | uniq -c
```

Explicație:

- ps -ef = listă completă a proceselor

- tail -n +2 = sare peste header
- awk '{print \$1}' = primul câmp (userul)
- sort | uniq -c = grupează și numără apariții

Sau pentru doar utilizatorii activi (logați):

```
who | awk '{print $1}' | sort -u | while read u; do echo "$u: $(ps -u $u | wc -l)"; done
```

Întrebarea 248

Q:

25. Scrieți un script UNIX Shell care calculează media fișierelor cu extensia ".txt" pe director din directorul curent și toate subdirectoarele sale.

A:

Script care calculează media fișierelor .txt pe director din directorul curent și subdirectoarele lui:

```
#!/bin/bash
total_txt=0
total_dir=0
for d in $(find . -type d); do
    total_dir=$((total_dir + 1))
    nr=$(ls "$d"/*.txt 2>/dev/null | wc -l)
    total_txt=$((total_txt + nr))
done
if [ $total_dir -gt 0 ]; then
    echo "Media: $((total_txt / total_dir)) fișiere .txt per director"
fi
```

Întrebarea 249

Q:

Câte procese va crea fragmentul de cod de mai jos, excluzând procesul părinte?

```
if(fork() != fork()) {
    fork();
}
```

A:

6 copii (7 procese inclusiv părintele).

Execuție:

- P face primul fork() → C1. În P: PID(C1), în C1: 0.
- P face al doilea fork() → C2.
- C1 face al doilea fork() → C3.
- Acum se compară valorile:
 - În P: PID(C1) != PID(C2) → true → fork() → C4
 - În C1: 0 != PID(C3) → true → fork() → C5

- În C2: $PID(C1) \neq 0 \rightarrow \text{true} \rightarrow \text{fork}() \rightarrow C6$
- În C3: $0 \neq 0 \rightarrow \text{false} \rightarrow \text{NU mai face fork}$

Total copii: C1, C2, C3, C4, C5, C6 = 6.

Întrebarea 250

Q:

27. Draw the hierarchy of processes generated by the following code: *nu va trb sa desenezi:))*

A:

(Notă din întrebare: "nu va trb sa desenezi")

Întrebare incompletă din arhiva originală — răspunsul depinde de codul care lipsește din enunț.

Întrebarea 251

Q:

28. Ce face apelul de sistem "write" când în PIPE există spațiu, dar nu suficient pentru cât i se cere să scrie?

A:

Scrie cât permite spațiul disponibil.

Întrebarea 252

Q:

30. Ce se întâmplă cu procesele zombie ale unui părinte care se termină?

A:

Sunt adoptate de procesul init (PID 1), care va apela wait pentru ele, eliberând astfel resursele lor. Astfel, procesele zombie nu mai rămân ca zombi indefinit.

Întrebarea 253

Q:

31. Planificați execuția joburilor de mai jos astfel încât suma întârzierilor să fie minimă: A/22/27, B/2/15, C/4/5.

(Format: Job/Durată/Termen)

A:

Ordinea: CBA — întârziere totală 1.

C: pornește la 0, termină la 4, termen 5 → întârziere 0

B: pornește la 4, termină la 6, termen 15 → întârziere 0

A: pornește la 6, termină la 28, termen 27 → întârziere 1

Întârziere totală = 1.

Întrebarea 254

Q:

33. Ce ați adăuga la următorul fragment de cod astfel încât să afișeze "1 3 3"? Modificările nu pot elimina din execuție liniile originale de cod.

```
int n = 0;
pthread_mutex_t m[3];
void* f(void* p) {
    int id = (int)p;
    pthread_mutex_lock(&m[id]);
    n += id;
    printf("%d ", n);
    pthread_mutex_unlock(&m[(id+1) % 3]);
    return NULL;
}

int main() {
    int i;
    pthread_t t[3];
    for (i=0; i<3; i++) pthread_mutex_init(&m[i], NULL);
    for (i=0; i<3; i++) pthread_create(&t[i], NULL, f, (void*)i);
    for (i=0; i<3; i++) pthread_join(t[i], NULL);
    for (i=0; i<3; i++) pthread_mutex_destroy(&m[i]);
    return 0;
}
```

A:

Pentru a afișa "1 3 3" în loc de "1 2 3" (output bazat pe ordinea creării thread-urilor),

trebuie să asiguri o ordine specifică de execuție. Soluție: bariere și/sau semnale.

Notă: întrebarea originală e incompletă din arhivă, dar conceptul cheie e:

- Folosește `pthread_barrier_wait` pentru a sincroniza thread-urile pe nivele
- SAU folosește `pthread_cond_wait` + `cond_signal` pentru ordine specifică
- SAU schimbă conținutul lui `printf` să folosească un index global

Răspunsul original cu OCR garbage nu poate fi recuperat. Pentru un răspuns complet,

ar fi util enunțul original al întrebării.

Întrebarea 255

Q:

38. Fișierul a.txt conține pe fiecare linie două numere separate prin spațiu. Scrieți o comandă UNIX Shell care afișează pentru fiecare linie suma numerelor sale.

A:

Comandă care afișează suma celor două numere de pe fiecare linie:

```
awk '{print $1 + $2}' a.txt
```

Explicație:

- \$1 = primul câmp
 - \$2 = al doilea câmp
 - print \$1 + \$2 = afișează suma
 - (separator default = spațiu/tab)
-

Întrebarea 256

Q:

37. Scrieți o comandă UNIX Shell care inversează în fișierul a.txt toate perechile de cifre vecine (ex: a3972b -> a9327b).

A:

```
sed -E "s/([0-9])([0-9])\2\1/gi" a.txt
```

```
sed 's/([[:digit:]]\)\([[:digit:]]\)\2\1/g' b.txt
```

2. sed -r 's/([0-9])([0-9])\2\1/g'

Întrebarea 257

Q:

Afișați doar liniile unui fișier a.txt care apar o singură dată (nu sunt duplicate).

A:

```
sort a.txt | uniq -u
```

Explicație:

- sort sortează liniile (necesar pentru uniq, care compară doar linii consecutive)
 - uniq -u afișează doar liniile care apar O SINGURĂ dată în fișier (-u = unique)
-

Întrebarea 258

Q:

40. Scrieți un script UNIX Shell care afișează numele fiecărui fișier .txt din directorul curent care conține cuvântul "cat".

A:

Script care afișează fișierele .txt din directorul curent care conțin "cat":

```
#!/bin/bash

for f in *.txt; do

    if [ -f "$f" ] && grep -q "cat" "$f"; then

        echo "$f"

    fi

done
```

Sau o singură linie folosind grep -l:

```
grep -l "cat" *.txt
```

Explicație:

- grep -q = quiet (nu afișează nimic, doar status code)
 - grep -l = afișează doar numele fișierului care match-uește
-

Întrebarea 259

Q:

41. În fragmentul de program de mai jos, marcați ce proces execută fiecare linie: Părintele, Copilul sau ambele.

```
k = fork();
if (k == 0) {
    printf("A\n");
} else {
    printf("B\n");
}
printf("C\n");
```

A:

Marcaj care proces execută fiecare linie:

```
k = fork(); // PĂRINTE (fork apelat de părinte)
if (k == 0) { // AMBELE (decizie executată în ambele)
    printf("A\n"); // COPIL (k=0 doar în copil)
} else { // AMBELE
    printf("B\n"); // PĂRINTE (k>0 doar în părinte)
}
printf("C\n"); // AMBELE (după if, ambele procese execută)
```

După fork(), AMBELE procese continuă de pe linia următoare. Diferența e doar valoarea lui k.

Întrebarea 260

Q:

42. Câte procese vor fi create de fragmentul de cod de mai jos, excluzând procesul părinte inițial?

```
fork(); wait(0); fork(); wait(0); fork();
```

A:

7 procese (excluzând părintele inițial).

Execuție:

P execută: fork → C1. wait(C1). fork → C2. wait(C2). fork → C3.

C1 execută: wait (returnează -1, nu are copii). fork → C4. wait(C4). fork → C5.

C2 execută: wait (-1). fork → C6.

C3 execută: (nimic, codul s-a terminat)

C4 execută: wait (-1). fork → C7.

C5 execută: (nimic)

C6 execută: (nimic)

C7 execută: (nimic)

Total copii creați: C1, C2, C3, C4, C5, C6, C7 = 7.

Întrebarea 261

Q:

43. Care sunt output-urile posibile în consolă ale următorului fragment de cod (ignorând orice output pe care l-ar putea genera `execl`), și când vor apărea?

```
printf("A\n");  
execl(...);  
printf("B\n");
```

A:

Dacă `execl` eșuează, va afișa "A" și "B".

Dacă `execl` reușește, va afișa doar "A" deoarece imaginea procesului curent va fi înlocuită cu una nouă.

Deci: "A\nB\n" când `execl` nu este executat sau eșuează; doar "A\n" în toate celelalte cazuri.

Întrebarea 262

Q:

44. Ce face apelul de sistem "read" când pipe-ul este gol?

A:

Va returna EOF (valoare returnată 0) dacă niciun proces nu mai are deschis capătul de scriere. Dacă alt proces are pipe-ul deschis pentru scriere, read va bloca în anticiparea unor date noi.

Întrebarea 263

Q:

45. Ce face apelul de sistem "open" înainte de a se întoarce când deschide un FIFO?

A:

Așteaptă până când capătul opus al FIFO-ului este deschis de alt proces. Apelul de sistem va aștepta până când operația complementară din celălalt proces este deschisă (deschidere pentru citire așteaptă scriere și viceversa).

Întrebarea 264

Q:

46. Dați un motiv pentru a alege thread-uri în loc de procese.

A:

Comunicarea între thread-uri este mai ușoară decât între procese din cauza datelor partajate.

- Mult mai rapid de creat un thread decât un proces.

- Mult mai rapidă schimbarea între thread-uri decât între procese.
- Comunicarea inter-thread este mai rapidă decât cea inter-proces deoarece thread-urile aceluiasi proces partajează memoria.

Dezavantaje:

- Un thread poate strica datele altui thread.
- Dacă un thread se blochează, toate thread-urile procesului se blochează.

Uneori e mai bine să folosești thread-uri în loc de procese deoarece thread-urile consumă mai puține resurse (au memorie partajată, folosesc variabile globale etc.), pe când procesele nu partajează date, consumă mai multe resurse și durează mai mult crearea și comunicarea între ele.

Întrebarea 265

Q:

47. Considerând că funcțiile "fa" și "fb" sunt rulate în thread-uri concurente, care va fi valoarea lui "n" după ce thread-urile sunt terminate? De ce?

```
pthread_mutex_t a, b;
int n = 0;

void* fa(void* p) {
    pthread_mutex_lock(&a);
    n++;
    pthread_mutex_unlock(&a);
}

void* fb(void* p) {
    pthread_mutex_lock(&b);
    n++;
    pthread_mutex_unlock(&b);
}
```

A:

Funcțiile fa și fb folosesc DOUĂ mutex-uri DIFERITE pentru aceeași resursă n.

Asta nu asigură excludere mutuală — un thread care apelează fa și unul care apelează fb pot intra în secțiunea critică SIMULTAN, modificând n în paralel → race conditions.

Valoarea finală a lui n este IMPREDICTIBILĂ — poate fi orice de la 0 la valoarea așteptată,

în funcție de cum se intercalează execuțiile.

Soluție corectă: TOATE thread-urile trebuie să folosească ACELAȘI mutex pentru resursa n.

Întrebarea 266

Q:

Job Duration Deadline

A 5 9

BO 7 13

Cc 12 10Schedule the following jobs so that they all meet at their deadlines A/5/9, B/7/13, C/1/10

A:

Ordinea ACB — toate joburile se încheie până la deadline (întârziere totală = 0).

Joburi:

- A: durată 5, deadline 9
- B: durată 7, deadline 13
- C: durată 1, deadline 10

Verificare ordine A, C, B:

- A: pornește la 0, termină la 5, deadline 9 → întârziere 0
- C: pornește la 5, termină la 6, deadline 10 → întârziere 0
- B: pornește la 6, termină la 13, deadline 13 → întârziere 0

Total întârziere: 0. Toate joburile la termen.

Întrebarea 267

Q:

49. Dați un avantaj și un dezavantaj al metodei de alocare segmentată față de metoda de alocare paginată.

A:

Dezavantaj: Metoda segmentată este mai lentă decât cea paginată.

Avantaj: Metoda segmentată permite partajarea procedurilor.

Alte avantaje:

- Tabelele de segmente folosesc mai puțină memorie decât paginarea.
- Nu oferă fragmentare internă, în timp ce alocarea paginată oferă.
- Mai simplu să relocăm segmente decât întregul spațiu de adrese.

Alte dezavantaje:

- Dimensiunea inegală a segmentelor nu este bună în cazul swap-ului.
- Necesită intervenția programatorului.
- Greu de alocat memorie contiguă deoarece are dimensiune variabilă.
- Algoritm costisitor de gestionare a memoriei.

Întrebarea 268

Q:

50. Când ați încărca în memorie paginile unui program care tocmai pornește?

A:

Când sunt solicitate. În momentul începerii execuției nu are nicio pagină în memorie (load la cerere / demand paging).

Întrebarea 269

Q:

51. Când își schimbă un proces starea din RUN în READY?

A:

Când un alt proces care era în starea WAIT din cauza unei operații I/O a terminat operația și este adus înapoi pentru execuție (sau când cuanta de timp a procesului curent expiră). Procesul curent este pus în READY, iar primul proces din coada Ready este pus să ruleze.

Întrebarea 270

Q:

52. Dat fiind un sistem de fişiere UNIX configurat cu un bloc de B octeţi care poate conţine A adrese, şi i-noduri cu S link-uri directe, un link de indirectare simplă, un link de indirectare dublă şi un link de indirectare triplă, daţi formula pentru dimensiunea maximă posibilă a unui fişier.

A:

Dimensiunea maximă = $(S + A + A*A + A*A*A) * B$ octeţi

Unde:

- $S * B$ = octeţi accesibili prin link-urile directe

- $A * B$ = octeţi prin indirectarea simplă (un bloc cu A adrese, fiecare către un bloc de date)

- $A * A * B$ = octeţi prin indirectarea dublă

- $A * A * A * B$ = octeţi prin indirectarea triplă

Întrebarea 271

Q:

67. Dat fiind un director şi toată ierarhia sa, scrieţi o comandă UNIX Shell care afişează toate subdirectoarele care conţin fişiere .c.

A:

```
find . -name "*.c" -printf "%h\n" | sort -u
```

Sau:

```
find . -type f -name "*.c" -exec dirname {} \; | sort -u
```

Întrebarea 272

Q:

68. Ce va afişa fragmentul de cod de mai jos în consolă?

```
int p[2];
char buf[10];
int n;
pipe(p);
n = read(p[0], buf, 10);
printf("%d\n", n);
```

A:

Nu va afişa nimic, deoarece va rămâne blocat la apelul read. Capătul de scriere nu a fost închis, deci read va aştepta date care nu vor veni niciodată.

Întrebarea 273

Q:

69. Scrieți o comandă UNIX Shell care afișează toate liniile dintr-un fișier a.txt care conțin cel puțin o distanță în kilometri.

A:

```
grep -E '[0-9]+\s*km\b' a.txt
```

Explicație:

- [0-9]+ = unul sau mai multe cifre
- \s* = zero sau mai multe spații (între număr și unitate)
- km = literal "km"
- \b = margine de cuvânt (ca să nu prindă "kml" sau "kmh")

Variante: `grep -E '\b[0-9]+\s*km\b' a.txt` (cu margini de cuvânt la ambele capete)

Întrebarea 274

Q:

70. Scrieți o comandă UNIX Shell care afișează liniile fișierului a.txt înlocuind orice perechi de vocală majusculă urmată de cifră pară (ex: a23E712BU4 -> a23E721B4U).

A:

```
sed -E 's/([AEIOU])([02468])\2\1/g' a.txt
```

Întrebarea 275

Q:

71. Scrieți o comandă UNIX Shell care afișează o listă a tuturor aparițiilor unice ale numelor de utilizator SCS aparținând secțiunilor engleză sau română care apar într-un fișier text a.txt.

A:

```
grep -oE '\b[a-z]+(_[eler])[0-9]+\b' a.txt | sort -u
```

(Presupunând că username-urile au sufix _e, _l, _r etc. pentru secțiuni.)

Întrebarea 276

Q:

76. Ce face apelul de sistem "write" când FIFO-ul conține mai puțin spațiu decât datele pe care i se cere să le scrie, dar nu este plin?

A:

Va scrie cât de multe date poate (cât permite spațiul disponibil).

Întrebarea 277

Q:

77. Dați operatorii pentru următoarele verificări Shell:

- șir nu e gol:
- este executabil:
- este diferit (pentru numere):
- este director:

A:

- șir nu e gol: -n (sau echivalent: ["\$x" != ""])

- este executabil: -x

- este diferit (numere): -ne

- este director: -d

Întrebarea 278

Q:

1. Scrieți o comandă UNIX Shell care afișează liniile dintr-un fișier care conțin cuvinte ce încep cu literă mare.

A:

grep -E '\b[A-Z][a-z]*\b' a.txt

Întrebarea 279

Q:

3. Fișierul c.txt conține pe fiecare linie două numere separate prin spațiu. Scrieți o comandă UNIX Shell care afișează pentru fiecare linie suma numerelor sale.

A:

awk '{print \$1 + \$2}' c.txt

Explicație:

- awk procesează fiecare linie automat
 - \$1 = prima coloană, \$2 = a doua coloană
 - print \$1 + \$2 = afișează suma lor
 - Implicit AWK consideră spațiul/tabul ca delimitator de coloane
-

Întrebarea 280

Q:

4. Display only the lines of a file d.txt that appear only once.

A:

sort d.txt | uniq -u

4. sort d.txt | uniq -u

Întrebarea 281

Q:

16. Când își schimbă un proces starea din RUN în READY?

A:

Când cuanta de timp a procesului curent expiră sau când task-ul se completează sau când e îndeplinită altă condiție, procesul curent este pus în coada Ready (dacă mai are instrucțiuni de executat) sau este marcat ca finalizat (dacă a terminat). În oricare caz, primul proces din coada Ready este scos și pus să ruleze.

Întrebarea 282

Q:

7. Câte procese sunt create de fragmentul de cod de mai jos, când procesul părinte P apelează f(3)?

```
void f(int n) {  
    if (n > 0 || fork() == 0) {  
        f(n-1);  
        exit(0);  
    }  
    wait(0);  
}
```

A:

```
void f(int n) {  
    if (n > 0 || fork() == 0) {  
        f(n - 1);  
        wait(NULL);  
    }
```

```
}  
}
```

Apelul `f(3)`: după ce `n` ajunge la 0, condiția `n>0` false → `fork() == 0` e evaluat.

Părintele: `fork()>0` → fals → NU intră în if. Continuă cu wait din apelul anterior.

Copilul: `fork()==0` → adevărat → intră în if cu `f(-1)`.

Pentru `f(-1)`: `n>0` false → `fork()` din nou. Părintele NU continuă. Copilul intră cu `f(-2)`.

Acest pattern continuă LA INFINIT – fork bomb!

Concluzie: programul NU SE TERMINĂ niciodată, creează un număr infinit de procese.

Întrebarea 283

Q:

5. Considerați că funcția `f` este executată simultan de 10 thread-uri. Ce ați adăuga pentru a asigura că valoarea lui `n` este 10 după ce thread-urile s-au terminat? În acest context, cum se numește linia "`n++`" și cum se numește variabila `n`?

```
pthread_mutex_t m;  
int n = 0;
```

```
void* f(void* p) {  
    pthread_mutex_lock(&m);  
    n++;  
    pthread_mutex_unlock(&m);  
    return NULL;  
}
```

A:

Codul deja are mutex, deci valoarea lui `n` va fi 10 dacă există 10 thread-uri care apelează `f`.

- Linia "`n++`" este într-o secțiune critică.

- Variabila `n` este o resursă partajată (shared variable).
